

Sisteme de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică

Email: paul.irofti@fmi.unibuc.ro
Web: <https://cs.unibuc.ro/~pirofti/so.html>

Octombrie, 2023

Curs

- ▶ Paul Irofti – Seria 23, Seria 25

Laborator:

- ▶ Horațiu Cheval – Seria 23
- ▶ Mihai Murgan (BitDefender) – Seria 251
- ▶ Nicolae Cristian Cătălin – Grupa 252

Laborator **30 puncte** – săptămânile 1–8 (aproximativ)

Punctaj

- ▶ **minim 15 puncte** pentru a intra în examen
- ▶ activitate 20 puncte ($\approx$ săptămânile 1–7)
- ▶ colocviu 10 puncte ($\approx$ în săptămâna 8)

Prezență

- ▶ **prezență obligatorie** la toate laboratoarele inclusiv colocviu
- ▶ fără prezență nu se intră în examen

Recuperare, Restanțe

- ▶ laboratorul trebuie promovat în timpul semestrului
- ▶ recuperarea doar în Semestrul I în timpul orelor de laborator
- ▶ nu se recuperează înainte de restanță sau în semestrul II

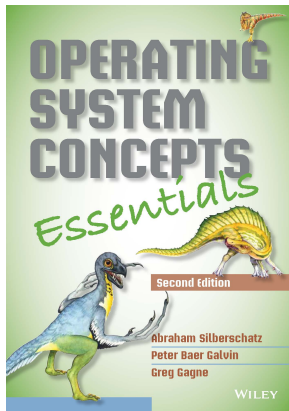
Proiect 20 puncte – săptămânile 9-14 (aproximativ)

- ▶ echipe de câte 2-3
- ▶ **punctaj identic** pentru fiecare membru
- ▶ ales dintr-o listă
- ▶ lucru la laborator după colocviu
- ▶ nu există notă minimă de promovare
- ▶ nu se ia în considerare prezența

Organizare – Examen

- ▶ Examen 50 puncte – **minim 25** puncte pentru promovare
- ▶ subiecte: probleme din materia de la curs
- ▶ este permis accesul la orice material didactic
- ▶ nu aveți voie cu dispozitive electronice!
- ▶ Minim 50 de puncte în total pentru promovare
- ▶ Exemplu: 25 examen + 25 laborator = 50p → 5 (cinci)
- ▶ Exemplu: 25 examen + 15 laborator = 40p → 4 (patru)
- ▶ chiar dacă promovați laboratorul și examenul, nota finală poate să nu fie de trecere!

Dinosaur Book



- ▶ A. Silberschatz, P.B. Galvin, and G. Gagne (2014). *Operating system concepts essentials*. John Wiley & Sons, Inc.
- ▶ Resurse:
<http://codex.cs.yale.edu/avi/os-book/OS10/>
- ▶ Distribuită gratuit în format PDF

În funcție de nivel, la curs vom acoperi următoarele capitole:

- ▶ Chapter 3 – Processes
- ▶ Chapter 4 – Threads & Concurrency
- ▶ Chapter 5 – CPU Scheduling
- ▶ Chapter 6 – Synchronization Tools
- ▶ Chapter 7 – Synchronization Examples
- ▶ Chapter 9 – Main Memory
- ▶ Chapter 10 Virtual Memory
- ▶ Chapter 13 File-System Interface
- ▶ Chapter 14 File-System Implementation

Vezi cuprinsul [aici](#).

În anii buni și în funcție de interesul generației voastre:

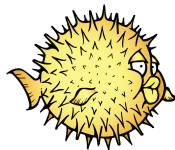
- ▶ Chapter 11 Mass-Storage Structure
- ▶ Chapter 12 I/O Systems
- ▶ Chapter 15 File-System Internals

Câteodată și următoarele capitole

- ▶ Chapter 16 Security
- ▶ Chapter 17 Protection

deși ele sunt acoperite la Masterul de [Security and Applied Logic](#) la cursul de [Operating Systems: Design and Security](#)

OpenBSD



- ▶ website: <http://www.openbsd.org/>
- ▶ source: <https://mirrors.pidginhost.com/pub/OpenBSD/6.9/src.tar.gz>
- ▶ github: <https://github.com/openbsd>
- ▶ source browsing: <https://cvsweb.openbsd.org/>
- ▶ manuals: <https://man.openbsd.org/>
- ▶ FAQ: <http://www.openbsd.org/faq/index.html>

Lucru în terminal:

- ▶ în orice sistem de operare, mai puțin Windows, aveți un terminal compatibil
- ▶ [Windows Subsystem for Linux](#) (WSL) – vine preinstalat de obicei în Windows 10
- ▶ [Cygwin](#) – emulator Linux pentru Windows
- ▶ Instalare SO tip UNIX în mașină virtuală ([VirtualBox](#) este open-source și gratuit)
- ▶ [Termux](#) – emulator pentru telefon
- ▶ [JSLinux](#) – emulator nativ web

Succes!

Întrebări?

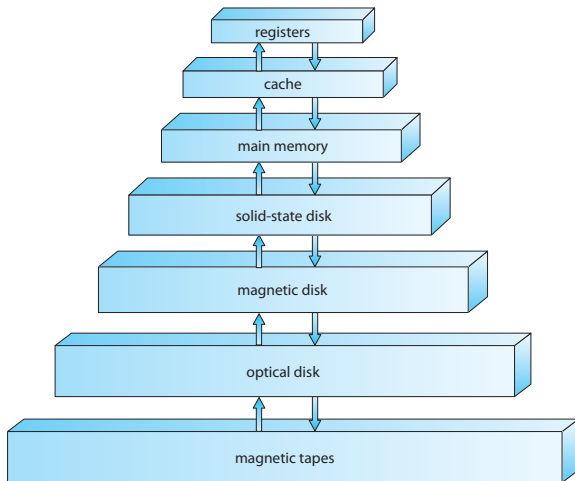
Sistemul de fișiere

Utilizarea Sistemelor de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică
Email: paul.irofti@fmi.unibuc.ro

Memoria

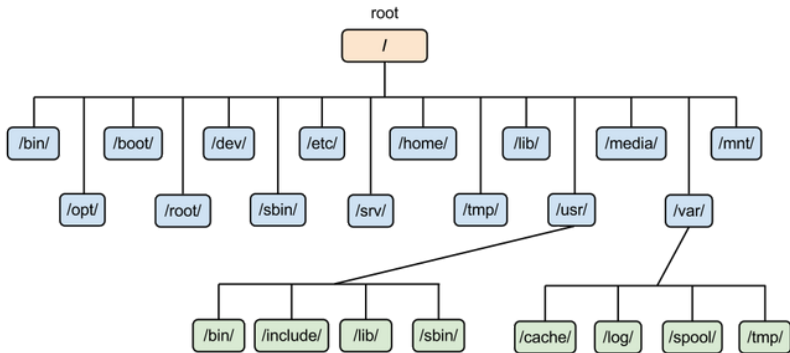


<http://codex.cs.yale.edu/avi/os-book/>

- ▶ stocare permanentă
- ▶ disponibile după întreruperea alimentării
- ▶ date folosite direct de utilizator
- ▶ salvate în memoria secundară (SSD → casete)

- ▶ un director conține fișiere
- ▶ informație organizată
- ▶ nu avem toate fișierele într-un loc (flat structure)
- ▶ un director poate conține alte directoare

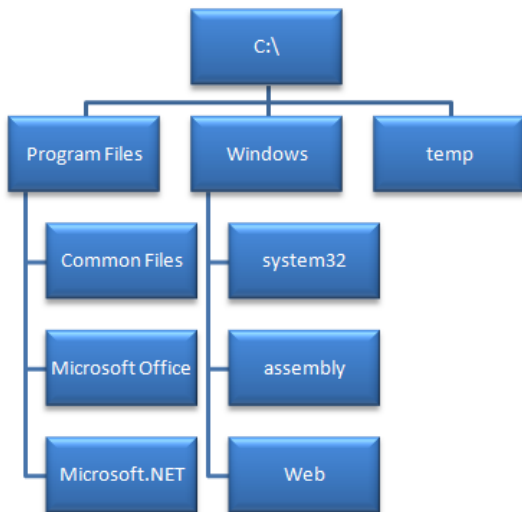
Unix



<https://nepalisupport.wordpress.com/>

- ▶ rădăcina este notată cu / și se mai numește și *root*.
- ▶ conține toate mediile de memorie
- ▶ fișierele în general nu au extensie
- ▶ separatorul folosit este /
- ▶ Exemplu: `/var/log/messages`

Windows



<https://linuxway.weebly.com/>

Windows: calea

- ▶ nu există o rădăcină unică
- ▶ fiecare entitate de stocare are rădăcina lui
- ▶ rădăcina este notată cu literă:
- ▶ indiferent de configurație tot timpul va exista C:
- ▶ C: conține toate fișierele necesare SO
- ▶ fișierele în general au extensie
- ▶ separatorul folosit este \ (invers decât Unix)
- ▶ versiunile mai noi de Windows acceptă și varianta Unix
- ▶ Exemplu: C:\Windows\System32

Lina de comandă

- ▶ flexibilitate
- ▶ automatizare
- ▶ control la distanță
- ▶ dificil la început
- ▶ mod de lucru: comandă-răspuns
- ▶ are la bază un program de tip `shell`
- ▶ promptul indică faptul că se așteaptă o comandă
- ▶ simbolizat de obicei prin
 - ▶ `$` sau `%` pentru utilizatorii comuni
 - ▶ `#` pentru administrator (denumit `root` în Unix).

- ▶ comanda în dreapta promptului
- ▶ rezultatul afișat fără a fi prefixat cu prompt
- ▶ încheierea comenzii → reapariția promptului

Example

```
$ echo " Hello , World!"  
Hello , World!  
$
```

- ▶ comandă necunoscută → citiți manualul
- ▶ manualul SO este împărțit în secțiuni
- ▶ intrări cu același nume dar în secțiuni diferite
- ▶ secțiuni
 1. general commands
 2. system calls and error numbers
 3. library functions
 4. device drivers

Example

```
$ man echo
$ man printf
$ man 3 printf
$ man man
```

Variabile de mediu

- ▶ dictează comportamentul programelor
- ▶ sunt setate de sistem sau utilizator
- ▶ scrise cu litere mari și precedate de simbolul \$
- ▶ la început terminalul vă plasează în directorul \$HOME
- ▶ terminalul caută comenzi în \$PATH
- ▶ pot fi setate sau modificate cu comanda export
- ▶ export este o comandă *internă* a shell-ului

- ▶ `pwd(1)` – print working directory
- ▶ `cd(1)` – change directory
- ▶ `ls(1)` – list directory contents
- ▶ `.` simbolizează directorul curent și `..` directorul părinte

Example

```
$ cd ../../../../..  
$ cd /home/paul  
$ cd $HOME  
$ cd
```


Redirecționare

- ▶ operatorul `>` redirecționează ieșirea în modul *create*
- ▶ operatorul `>>` redirecționează ieșire în modul *append*
- ▶ operatorul `<` redirecționează intrarea
- ▶ operatorul `|` (denumit și *pipe*): ieșirea comenzii din stânga devine intrarea celei din dreapta

Example

```
$ echo "lorem ipsum" > foo  
$ echo "more lorem" >> foo  
$ echo "less ipsum" > foo
```

Cum va arăta foo la fiecare pas?

- ▶ `cat(1)` – pentru fișiere text scurte
- ▶ `less(1)` – pentru fișiere text lungi
- ▶ `hexdump(1)` – pentru fișiere binare

Example

```
$ cat foo
less ipsum
$ less /etc/passwd # exit with 'q'
$ which ls
/bin/ls
$ hexdump -C 'which ls' | less
```

Wildcards

- ▶ ^ – simbolizează începutul liniei
- ▶ \$ – simbolizează sfârșitul liniei
- ▶ un atom este un caracter sau un set de caractere grupat cu [] sau ()
- ▶ * – găsește de 0 sau mai multe ori atomul precedent
- ▶ + – găsește de 1 sau mai multe ori atomul precedent
- ▶ ? – găsește de 0 sau 1 ori atomul precedent

Căutarea în fișiere

- ▶ `grep(1)` – comanda cea mai folosită de căutare
- ▶ folosește tipare și expresii regulate
- ▶ uz tipic: `grep expresie fișiere`
- ▶ opțiuni utile
 - ▶ `-R` – căutare recursivă (intră în toate subdirectoarele)
 - ▶ `-i` – case insensitive
 - ▶ `-n` – indică numărul liniei pe care a fost găsită expresia

Example

```
$ grep printf *.c  
$ grep printf *. [ch]
```

Căutarea în directoare

- ▶ `find(1)` – comanda cea mai folosită de căutare
- ▶ uz tipic: `find cale -name expresie`
- ▶ opțiuni utile
 - ▶ `-exec` – execută comanda asupra elementelor găsite
 - ▶ `-type` – limitează rezultatele la un anumit tip (ex. fișiere)
 - ▶ `-iname` – la fel ca `-name` dar case insensitive
 - ▶ `-maxdepth` – nivelul de recursivitate

Example

```
$ find / -name echo # foarte lent!  
$ find . -iname *.o -exec rm {} \;
```

Manipulare

- ▶ `mkdir(1)` – crează directoare
- ▶ `rmdir(1)` – șterge directoare fără conținut
- ▶ `touch(1)` – crează un fișier gol dacă nu există deja
- ▶ `mv(1)` – mută directoare sau fișiere
- ▶ `cp(1)` – copiază directoare (cu `-r`) sau fișiere
- ▶ `rm(1)` – șterge directoare (cu `-r`) sau fișiere

Expresii regulate

- ▶ `[0-9]+ [a-z]+ [A-Z]+ [a-zA-Z0-9]+`
- ▶ `[:alnum:] [:cntrl:] [:lower:] [:space:]`
- ▶ `[:alpha:] [:digit:] [:print:] [:upper:]`
- ▶ am fosolit deja o expresie regulată? când?

Example

```
$ egrep [a-z]+[0-9]+ *.tex
\usepackage[utf8]{inputenc}
\item ... Windows\textbackslash System32}
$ egrep '\([0-9]+' *.tex
\item \code{pwd(1)} — print ...
\item \code{cd(1)} — change ...
\item \code{ls(1)} — list ...
\item \code{cat(1)} — pentru ...
...
```

Virtualizare

Utilizarea Sistemelor de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică
Email: paul.irofti@fmi.unibuc.ro

Hardware (1)

- ▶ carcasa (chasis) poate susține anumite plăci de bază și alimentări
- ▶ alimentarea: power supply unit (PSU) – definită de numărul de wați
- ▶ placa de bază: motherboard
 - ▶ program de start: basic input output system (BIOS)
 - ▶ sau Unified Extensible Firmware Interface (UEFI)
 - ▶ include mai multe module hardware și asigură comunicația între ele

Hardware (2)

- ▶ procesorul: central processing unit (CPU)
- ▶ memoria de sistem: random access memory (RAM)
- ▶ controller disk: PATA, SATA, SCSI
 - ▶ conectează: hard drive, cdrom, usb etc.
- ▶ periferice: Peripheral Component Interconnect (PCI)
 - ▶ conectează: placa video, audio, de rețea etc.

- ▶ un disk poate avea mai multe partiții
- ▶ o partiție este o entitate de sine stătătoare (apare ca un "disk")
- ▶ fiecare disk începe cu master boot record (MBR)
- ▶ disk-urile pot avea 4 partiții fizice (salvate în MBR, hardware)
- ▶ există și partiții logice (în software)
- ▶ partițiile sunt folosite pentru organizare (similar cu directoarele ca scop)

Partiții fizice

Example

Disk: sd0 geometry: 29185/255/63 [468862128 Sectors]
Offset: 0 Signature: 0xAA55

#:	id	Starting			Ending			LBA Info:	start:	size	]
		C	H	S	C	H	S				
0:	A6	0	1	2	13054	43	45	64:	100G]	OpenBSD	
*1:	07	13054	75	14	13118	9	11	209717248:	0G]	NTFS	
2:	07	13118	9	12	29075	133	2	210741248:	122G]	NTFS	
3:	27	29075	155	9	29184	251	59	467099648:	1G]	Win Recovery	

Patru partiții fizice de tipuri diferite: OpenBSD, NTFS, Win Recovery. Partiția cu SO (activă) este partiția numărul 1.

Partiții logice

Example

```
$ df -h
```

Filesystem	Size	Used	Avail	Capacity	Mounted on
/dev/sd0a	1005M	388M	567M	41%	/
/dev/sd0d	3.9G	8.4M	3.7G	0%	/tmp
/dev/sd0f	2.0G	1.5G	399M	79%	/usr
/dev/sd0g	1005M	218M	737M	23%	/usr/X11R6
/dev/sd0h	9.8G	7.4G	1.9G	79%	/usr/local
/dev/sd0j	2.0G	195M	1.7G	10%	/usr/obj
/dev/sd0i	2.0G	1.3G	540M	72%	/usr/src
/dev/sd0e	11.2G	314M	10.3G	3%	/var
/dev/sd1a	61.7G	54.1G	4.5G	92%	/home
fusefs	7.6G	72.6M	7.5G	1%	/mnt

Pe partiția fizică 0 există alte 8 partiții logice. În plus se pot vedea alte 2 disk-uri montate.

Mașina virtuală

- ▶ virtualizează mediul hardware în software
- ▶ inclusiv CPU, memorie și disk
- ▶ poate executa un sistem de operare fără ca acesta să fie modificat în vreun fel
- ▶ o instanță unei mașini virtuale devine astfel un proces ca oricare altul
- ▶ împarte resursele mașinii reale (gazdă) cu celelate mașini virtuale și procese

- ▶ host – sistemul obișnuit, gazda
- ▶ hypervisor – soluția de virtualizare
- ▶ guest – mașina virtuală efectivă

- ▶ folosește direct hardware-ul
- ▶ reprezintă sistemul pe care se execută soluția de virtualizare
- ▶ de obicei este alcătuit din hardware și sistemul de operare
- ▶ există și soluții pur hardware (ex. Xen, SPARC)

Hypervisor

- ▶ soluția de virtualizare
- ▶ oferă o interfața de acces la hardware pentru guest (sistemul virtualizat)
- ▶ crează componente virtuale: CPU virtual, disk virtual, RAM virtual etc.
- ▶ pot exista unul sau mai mulți hypervisoari pe un host
- ▶ se mai numește și *virtual machine manager* (vmm)
- ▶ există soluții hardware și software
- ▶ exemple software: VirtualBox, QEMU, VMWare

- ▶ mașina virtuală efectivă
- ▶ execută sistemul de operare virtualizat
- ▶ sistem complet cu ajutorul hardware-ului virtualizat de hypervisor
- ▶ pot exista unul sau mai multe sisteme guest pe un host (o gazdă poate avea mai mulți oaspeți deodată)
- ▶ instalarea și utilizarea sistemului de operare virtualizat sunt (aproape) identice cu cele de pe un sistem hardware obișnuit

Resurse pentru instalarea Ubuntu într-o mașină virtuală

- ▶ Host: Windows, MacOS sau Linux
- ▶ Hypervisor: VirtualBox <https://www.virtualbox.org/>
- ▶ Guest: Ubuntu <https://www.ubuntu.com/> (Desktop, 1.5G)

Datele asociate unei mașini virtuale

- ▶ fișier asociat hard-disk-ului virtual (numit și imagine)
 - ▶ VirtualBox: fișier în format `.vdi`
 - ▶ QEMU: fișier în format `.qcow2`
 - ▶ VMWare: fișier în format `.vmdk`
- ▶ fișier cu specificații hardware ce definesc mediul de execuție al mașini virtuale
 - ▶ VirtualBox: fișier în format XML de tip `.vbox`
 - ▶ QEMU: opțiuni în linia de comandă (există soluții externe care oferă funcționalitate pentru fișiere de configurare XML)
 - ▶ VMWare: fișier în format cheie-valoare de tip `.vmx`
- ▶ opțional: toate aceste date sunt conținute într-un director

Care sunt avantajele imediate oferite de reprezentarea VM?

- ▶ pornește de la o imagine de bază (ex. instalare proaspătă Ubuntu)
- ▶ peste care efectuează modificări (ex. configurare server web)
- ▶ modificările sunt înmagazinate separat: fie în alt fișier fie într-o secțiune separată în fișierul imagine existent
- ▶ astfel putem face operații de tip *undo* și *redo*
- ▶ sau putem refolosi imaginea de bază pentru adăugarea altui serviciu (ex. configurarea unui server de baze de date)

- ▶ mașinile virtuale se importă/exportă din VirtualBox și VMWare; în QEMU se copiază pur și simplu fișierul asociat hard-disk-ului
- ▶ există format comun pentru a migra de la o soluție VM la alta: Open Virtualization Format (OVF)

Mediul de programare

Utilizarea Sistemelor de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică
Email: paul.irofti@fmi.unibuc.ro

De la sursă la executabil

- ▶ sursa – fișierul scris în limbaj de programare (ex. C, C++)
- ▶ compilator – traduce fișierul sursă în limbaj mașină (de asamblare)
- ▶ obiect – fișier rezultat din compilarea unui fișier sursă
- ▶ bibliotecă – fișier binar (deja compilat) ce oferă o anumită funcționalitate (ex. funcții de comunicare în rețea)
- ▶ linker – leagă obiecte și biblioteci pentru a produce executabilul
- ▶ executabil static – toate obiectele și bibliotecile se regăsesc în fișierul executabil rezultat
- ▶ executabil dinamic – conține doar informațiile și instrucțiunile proprii; bibliotecile sunt păstrate în fișiere separate

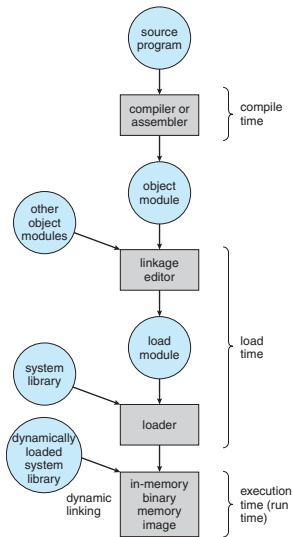
Rezolvarea dependențelor executabilelor dinamice la momentul încărcării în memorie:

1. sistemul de operare încarcă executabilul în memorie
2. înainte de a fi executat se caută bibliotecile folosite
3. se încarcă toate bibliotecile necesare
4. în caz că nu se găsesc una sau mai multe biblioteci executabilul este scos din memorie și execuția este anulată

Rezolvarea dependențelor executabilelor dinamice la nevoie:

1. sistemul de operare încarcă executabilul în memorie
2. înainte de a fi executat se caută bibliotecile strict necesare lansării programului
3. pe parcursul execuției dacă este nevoie de o bibliotecă
 - 3.1 execuția programului este oprită
 - 3.2 se caută în sistem biblioteca necesară
 - 3.3 se încarcă în memorie
 - 3.4 se repornește de unde a rămas execuția programului
4. în orice moment dacă o bibliotecă nu este găsită și încărcată executabilul este scos din memorie și execuția este anulată

Compiler



<http://codex.cs.yale.edu/avi/os-book/>

În UNIX există două soluții principale:

- ▶ gcc – GNU Compiler Collection (C, C++, Java etc)
- ▶ clang – LLVM front-end pentru C, C++, Objective C, OpenCL etc.

Indiferent de compilator, în UNIX există comanda `cc` (C compiler) și, opțional, comanda `c++` (C++ compiler).

Executabil simplu

În forma cea mai simplă compilatorul

- ▶ primește ca argumente fișierele sursă
- ▶ produce executabilul a.out
- ▶ denumit din motive istorice: formatul de executabil standard pe prima versiune UNIX

Example

```
$ ls
hello.c
$ cc hello.c
$ ls
a.out    hello.c
$ ./a.out
Hello , World!
```

Pentru execuție prefixăm a.out cu ./ pentru a spune shell-ului să nu caute executabilul în \$PATH ci în directorul curent.

- ▶ se folosește opțiunea `-c`
- ▶ compilatorul se oprește după producerea obiectului
- ▶ nu continuă cu crearea unui executabil
- ▶ obiectul va fi folosit împreună cu alte obiecte și biblioteci pentru a produce executabilul final

Example

```
$ ls  
hello.c  
$ cc -c hello.c  
$ ls  
hello.o    hello.c
```

Pentru a folosi biblioteci externe

- ▶ se folosește opțiunea `-lname` (**fără spații!**)
- ▶ compilatorul caută biblioteca într-o listă de directoare
- ▶ lista este de regulă generată și întreținută de comanda `ldconfig(1)`
- ▶ poate fi folosită variabila de mediu `$LD_LIBRARY_PATH` pentru a adăuga directoare proprii ce conțin biblioteci
- ▶ biblioteca găsită este folosită împreună cu alte obiecte și biblioteci pentru a produce executabilul final

Example

Folosirea bibliotecii de sistem *crypto*:

```
$ cc hello.c -lcrypto
```

Opțiuni utile

Prezentăm aici câteva opțiuni utile folosite des la compilare

- ▶ `-o nume` – numele executabilului sau obiectului rezultat (ex. `hello` înloc de `a.out`)
- ▶ `-Wall` – afișează toate mesajele de *warning*
- ▶ `-g` – pregătește executabilul pentru o sesiune de depanare (debug)
- ▶ `-Onumăr` – nivelul de optimizare (`0` → fără optimizări)
- ▶ `-O2` este folosit implicit, `-O0` este folosit adesea împreună cu `-g` pentru depanare

Example

Compilează `hello.c` cu simboluri de debug, fără optimizare, și numește executabilul `hello`

```
$ cc -g -O0 hello.c -o hello
```


Noțiuni folosite într-o sesiune de depanare

- ▶ break, breakpoint
 - ▶ punct în care să se oprească execuția
 - ▶ poate fi o funcție, o linie dintr-un fișier, chiar și o instrucțiune de asamblare
 - ▶ se poate investiga starea variabilelor la momentul respectiv
- ▶ backtrace, trace, stack trace
 - ▶ apelul de funcții care a dus în punctul curent
 - ▶ exemplu: `main()` → `decompress()` → `zip_decompress()`
- ▶ watch, watchpoint
 - ▶ punct în care să se oprească execuția **dacă este îndeplinită o condiție cerută de utilizator**
 - ▶ poate fi modificarea unei zone de memorie sau a unei variabile

Programul cel mai comun folosit pentru depanarea executabilelor este `gdb(1)`.

Comenzi uzuale

- ▶ `break simbol` – setează breakpoint la simbolul respectiv (ex. `main()` sau `hello.c:10`)
- ▶ `run` – începe execuția (de regulă după ce au fost setate breakpoint-urile)
- ▶ `next` – o dată ajuns într-un punct de oprire, mergi la următoarea instrucțiune
- ▶ `continue` – continuă până la următorul punct de oprire sau până la terminarea execuției programului
- ▶ `print variabilă` – afișează valoarea unei variabile
- ▶ `quit` – ieșire din gdb(1)

Example

```
$ cc -g -O0 hello.c -o hello
$ gdb hello
(gdb) break main
Breakpoint 1 at 0x546: file hello.c, line 5.
(gdb) run
Starting program: /home/paul/wrk/ub/uso/curs/3/hello
Breakpoint 1 at 0x101a49c00546: file hello.c, line 5.

Breakpoint 1, main () at hello.c:5
5          printf(" Hello , World!");
(gdb) next
Hello , World!
6          return 0;
(gdb) continue
Continuing.
Program exited normally.
(gdb) quit
```

Produsele software sunt alcătuite din mai multe componente:

- ▶ diferite module (ex. comunicație, logging)
- ▶ diferite biblioteci (ex. compresie, criptografie, http)
- ▶ mai multe fișiere sursă (numite și unități de compilare)
- ▶ între toate acestea apar diferite dependențe (ex. modulul de comunicație depinde de biblioteca http)
- ▶ inter-dependențele dictează ordinea de compilare
- ▶ dependențele și ordinea de compilare este scrisă formal în fișiere de tip Makefile
- ▶ instrucțiunile dintr-un Makefile sunt executate cu ajutorul comenzi `make`(1)

Format fix

```
target: dependency1 dependency2 [...]  
      commands
```

- ▶ target – ce va fi produs
- ▶ dependency – ingredientele necesare (deja existente)
- ▶ commands – comenzile pentru a produce target-ul
- ▶ **Atenție!** – este un TAB înaintea commands
- ▶ primul target din fișier sau, dacă există, target-ul all vor fi executate implicit

Avantaj: recompilează doar fișierele modificate!

Makefile simplu

Example

```
$ cat Makefile
all: hello
hello: hello.c
        cc -o hello hello.c
clean:
        rm hello

$ make
cc -o hello hello.c
$ make clean
rm hello
$ make hello
cc -o hello hello.c
```

Variable Makefile

Câteva variabile utile

- ▶ `$@` – numele target-ului curent (partea stângă)
- ▶ `$$` – toate dependențele (partea dreaptă)
- ▶ `$<` – numele primei dependențe (ex. `dependency1`)
- ▶ `.type1.type2:` – target-ul transformă fișiere de tip 1 în fișiere de tip 2 (ex. `.c.o:`)
- ▶ `$(VAR:old=new)` – înlocuiește `old` cu `new` în variabilă

Makefile complex

Example

```
PROG=hello
SRCS=hello.c
OBJS=$(SRCS:.c=.o)
CC=cc
CFLAGS=-g -O0
INSTALL=install

all: $(PROG)

$(PROG): $(OBJS)
        $(CC) -o $@ $^ $(CFLAGS)

.c.o:
        $(CC) -c -o $@ $< $(CFLAGS)

install: $(PROG)
        $(INSTALL) $< ${HOME}/bin

clean:
        -rm $(PROG) $(OBJS)
```


Utilizatori și procese

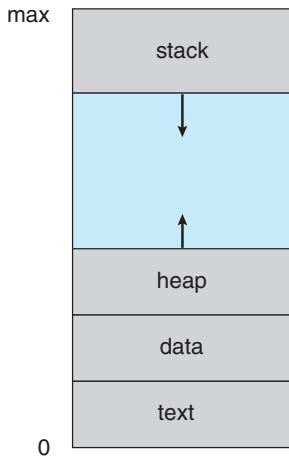
Utilizarea Sistemelor de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică
Email: paul.irofti@fmi.unibuc.ro

- ▶ un proces reprezintă un program în execuție
- ▶ el este însoțit de date suplimentare față de codul asociat programului
 - ▶ program counter
 - ▶ regiștri
 - ▶ date globale
 - ▶ stiva – include parametrii funcției curente, adresa de întoarcere, variabile locale etc.
- ▶ un program devine proces când este încărcat în memorie

Structura unui proces



<http://codex.cs.yale.edu/avi/os-book/>

- ▶ un actor în cadrul sistemului de operare
- ▶ nu neapărat un actor uman
- ▶ folosit în scopuri de organizare și separare a datelor și proceselor
- ▶ un utilizator poate avea mai multe procese în execuție
- ▶ un sistem de operare poate avea mai mulți utilizatori activi concomitent
- ▶ **probleme:** securitate, intimitate, acces (echitabil) la resurse

- ▶ parolă
- ▶ cheie criptografică
- ▶ cu ajutorul mai multor factori

Autentificare – parolă

Pereche user:pass

- ▶ fiecare utilizator are asociată o parolă de acces
- ▶ cel mai vechi și des întâlnit mecanism de autentificare
- ▶ parolele trebuie să fie schimbate des (6 luni)
- ▶ evitați parole scurte și simple:
 - ▶ un singur cuvânt (atac de dicționar)
 - ▶ data nașterii sau alte date personale (găsite în conturi și acte publice)
 - ▶ parole care conțin doar litere sau doar cifre (atac brute-force)
- ▶ comandă în Unix: `passwd(1)`
- ▶ fișiere: `/etc/passwd`, `/etc/master.passwd`
- ▶ parola este salvată criptografic (ex. folosind o funcție de hashing)
- ▶ folosită în URI: `https://user:pass@mail.fmi.unibuc.ro/`

Autentificare – cheie criptografică

Criptografie asimetrică

- ▶ autentificare cu ajutorul a două chei: una publică și una privată
- ▶ cheia publică poate fi distribuită tuturor (ex. homepage)
- ▶ cheia publică este preluată de terți și instalată pe sistemele la care vor să vă dea acces
- ▶ cheia privată este folosită împreună cu cheia publică pentru autentificare și acces
- ▶ cheia privată poate fi însoțită de o parolă (2FA)
- ▶ cheia privată trebuie păstrată în siguranță
- ▶ dacă cheia privată a fost compromisă, trebuie creată o nouă pereche de chei iar cea publică trebuie înlocuită pe toate sistemele
- ▶ comandă în Unix: `ssh-keygen(1)`
- ▶ fișiere: `~/.ssh/id_{dsa,ecdsa,ed25519,rsa} [.pub]`

Two Factor Authentication (2FA)

- ▶ autentificare folosind doi factori (ex. ATM: card și PIN)
- ▶ pe calculator se folosește de regulă parola și un număr (PIN)
- ▶ PIN-ul poate fi trimis după autentificarea cu parolă prin SMS
- ▶ PIN-ul poate fi un număr pseudo-aleator generat de un token
- ▶ token-ul poate fi o aplicație de telefon (ex. Authy) sau un dispozitiv separat (folosit în general de bănci)
- ▶ pe laptop și telefoane mobile se pot folosi senzori biometrici (ex. amprenta) pentru al doilea factor

- ▶ separarea accesului la fișiere și directoare între utilizatori
- ▶ drepturi de acces primare: citire, scriere, execuție
- ▶ cine aribtrează separarea?
- ▶ o soluție: fiecare director și fișier aparțin unui singur utilizator
- ▶ rezultat: lucrul în echipă și distribuirea datelor devine dificil
- ▶ îmbunătățire: grupuri de utilizatori care au acces

- ▶ mai mulți utilizatori pot aparține unui grup
- ▶ un utilizator poate aparține mai multor grupuri
- ▶ drepturi de acces la nivel de grup
- ▶ exemplu: grupul `staff` pentru cei ce administrează sistemul
- ▶ comandă în Unix: `groupadd(1)`, `usermod(1)#-G`
- ▶ exemplu: `groupadd gr151 && usermod alex -G gr151`
- ▶ fișiere: `/etc/group`, `/etc/passwd`

Fișierele și directoare au trei tipuri de acces pentru trei categorii

- ▶ acces: citire (r), scriere (w), executare (x)
- ▶ reprezentați prin biți (ex. 000→---, 110→rw-, 101→r-x)
- ▶ categorii: utilizator (u), grup (g), restul (o)
- ▶ `ls(1)#-l` codifică `drwxrwxrwx`
 - ▶ prima literă spune dacă obiectul este director sau nu
 - ▶ urmată de grupuri de acces pentru utilizator, grup și restul
- ▶ directoarele trebuie să aibă x ca să poată fi parcurse
- ▶ `-r--r--r--` – un fișier cu drepturi de citire pentru toată lumea
- ▶ `dr-xr-xr--` – un director în care pot intra doar utilizatorul și cei din grup
- ▶ ce se întâmplă dacă vrem să dăm acces la mai mult de un grup?
- ▶ comenzi: `chown(1)`, `chmod(1)`

Intimitate – Procese

- ▶ separarea accesului la procese între utilizatori
- ▶ programele sunt executate de utilizatori → procese
- ▶ procesele sunt deținute de utilizatorul care le-a pornit
- ▶ procesele au acces la resursele sistemului în funcție de privilegiile utilizatorului
- ▶ un proces fiu creat de alt proces moștenește drepturile părintelui
- ▶ Unix: un proces poate avea dreptul de a-și schimba utilizatorul și grupul de care aparține: `setuid` și `setgid`
- ▶ procesele unui utilizator nu pot fi oprite, pornite sau manipulate în alt mod de alți utilizatori
- ▶ excepție: administratorii sistemului
- ▶ comandă în Unix: `ps(1)`, `top(1)`, `kill(1)`

- ▶ un proces poate influența execuția altora prin modul în care folosește resurse
- ▶ pe un procesor poate fi activ un singur proces la un moment dat
- ▶ ordinea în care intră pe procesor un proces este dictată de algoritmul de scheduling din sistemul de operare (numit și scheduler)
- ▶ scheduler-ul ordonează execuția în funcție de prioritatea (ponderea) asignată proceselor
- ▶ Unix: prioritatea proceselor obișnuite este între -20 (cea mai mare) și 20 (cea mai mică)
- ▶ comandă în Unix: `nice(1)`, `renice(8)`

Administrație și comenzi

- ▶ cod de administrator: `root` în Unix, `administrator` în Windows
- ▶ utilizator cu drepturi comune pentru activitățile de zi cu zi
- ▶ apel la contul de administrație doar când este cazul
 - ▶ instalare de software
 - ▶ actualizarea sistem de operare
 - ▶ modificare fișiere de configurare etc.
- ▶ comenzi Unix: `su(1)`, `sudo(1)`

- ▶ `su(1)` – substitute user
- ▶ implicit execută un shell drept root (#)
- ▶ cere parola înainte de a schimba utilizatorul
- ▶ la ieşire revine la shell-ul utilizatorului original
- ▶ `su alex` – porneşte un shell drept utilizatorul alex
- ▶ `su -c cmd` – execută `cmd` fără a intra în shell
- ▶ `su alex -c cmd` – execută `cmd` drept utilizatorul alex

sudo – substitute user and do

- ▶ funcționalitate similară cu `su#-c`
- ▶ bazat pe un set de reguli ce includ
 - ▶ ce utilizatori pot apela comanda (implicit cei din grupul `wheel`)
 - ▶ au nevoie de parolă pentru execuție?
 - ▶ implicit e nevoie de parola utilizatorului curent
 - ▶ restricționarea la un set specific de comenzi ce sunt permise (ex. alex execută doar `ls(1)` ca root)
- ▶ set de reguli scris în `/etc/sudoers`
- ▶ editat cu `visudo(8)` – `vi(1)` specializat pe format `sudo(8)` ce verifică erori înainte de a modifica configurația
- ▶ ce face comanda: `sudo su?`

Administrare Unix – comenzi uzuale

- ▶ `useradd(8)` – adăugare utilizator (`adduser(8)` – versiunea interactivă)
- ▶ `userdel(8)` – ștergere utilizator
- ▶ `usermod(8)` – modificare date utilizator (`$HOME`, grupuri, `$SHELL`)
- ▶ `groupadd(8)` – adăugare grup
- ▶ `groupdel(8)` – ștergere grup
- ▶ `groupmod(8)` – modificare grup (schimbare nume, id)
- ▶ `vipw(8)` – editor `vi(1)` specializat pentru fișierul `passwd`
- ▶ `chpass(8)` – schimbare date utilizator (nume, telefon, când să schimbe parola)
- ▶ `id(1)`, `finger(1)`, `who(1)`, `w(1)`, `last(1)` – informații despre utilizatori și activitatea lor recentă

Administrare Unix – nomenclatură

- ▶ `pid` – process identification
- ▶ `ppid` – parent process identification
- ▶ `tid` – thread identification
- ▶ `uid` – user identification
- ▶ `gid` – group identification
- ▶ `res` – resident memory (cât ocupă în memoria principală)
- ▶ `pri` – prioritatea procesului
- ▶ `state` – starea procesului (gata de execuție, în așteptare, zombie, în execuție etc.)
- ▶ `init` – primul proces în sistem (`pid= 1`)
- ▶ `wchan` – numele funcției în care procesul e suspendat așteptând evenimente sau date noi

Lucrul în rețea

Utilizarea Sistemelor de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică
Email: paul.irofti@fmi.unibuc.ro

Intranet sau rețea

- ▶ alcătuit din noduri (calculatoare) numite *host-uri*
- ▶ legături fizice: cabluri de rețea, canale wireless
- ▶ legături logice: calea prin intermediul legăturilor fizice dintre un nod sursă și unul destinație
- ▶ probleme: parcurgere de graf, calea cea mai scurtă, comis-voiajor
- ▶ o rețea poate fi conectată la una sau mai multe rețele

Internet: set de rețele publice interconectate

Modul de comunicare în rețea se desfășoară urmând unul sau mai multe protocoale

- ▶ descrie în documente *Request for Comments* (RFC)
- ▶ Internet Engineering Task Force (IETF)
- ▶ Internet Society (ISOC)
- ▶ comitete formate din ingineri și informaticieni
- ▶ <https://www.rfc-editor.org/retrieve/>
- ▶ exemple: IP, TCP, DNS, SMTP, IMAP, POP, FTP, SSH
- ▶ majoritatea construite peste TCP și IP
- ▶ **atenție:** protocolul este implementat, nu reprezintă o implementare

Internet Protocol (IP)

Fiecare host este identificat printr-una sau mai multe adrese IP

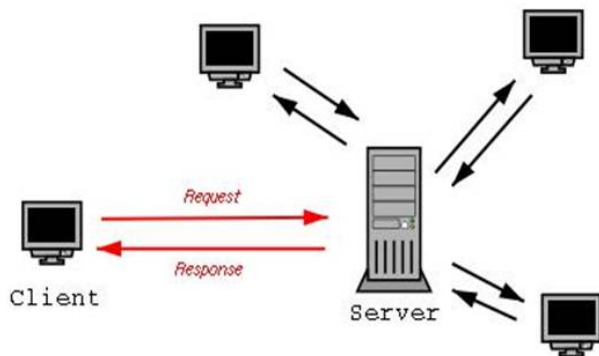
- ▶ versiunea inițială (IPv4) folosește 32-biți (RFC791)
- ▶ versiunea curentă este pe 128-biți (IPv6)
- ▶ IPv4 în continuare cea mai folosită
- ▶ 32 de biți reprezintă 4 bytes care sunt grupați separat prin .
- ▶ interval: 0.0.0.0 – 255.255.255.255
- ▶ anumite subintervale sunt folosite pentru adresare privată
- ▶ restul adreselor publice sunt accesibile de orice nod din rețea
- ▶ **exemplu:** 192.168.1.1, 10.0.0.1, 216.58.214.238, 8.8.8.8

Domain Name System (DNS)

Numele asociat unei adrese IP

- ▶ mai ușor de ținut minte pentru utilizatorii umani
- ▶ protocol descris în RFC1035
- ▶ dacă o adresa IP se schimbă, numele rămâne
- ▶ inițial toată lumea ținea o agendă locală în `/etc/hosts`
- ▶ astăzi procesul e centralizat
- ▶ toată lumea folosește aceeași agendă
- ▶ mai multe *host*-uri, numite *name server*, care conțin o copie a agendei
- ▶ serverele principale de la care copiază informația restul se numesc *root name servers*
- ▶ **exemplu:** `fmi.unibuc.ro` → `193.226.51.6`
- ▶ comenzi în Unix: `nslookup(1)`, `whois(1)`, `dig(1)`

Client-Server



<http://abcnetworking.wikispaces.com>

Comunicarea client-server se face prin *sockets*

- ▶ adresă: adresa IP a unui *host*
- ▶ port: intrare a *host*-ului
- ▶ socket: definește o posibilă conexiune IP:port
- ▶ o conexiune se face cu doi socketi și un protocol comun
- ▶ un socket pentru server și unul pentru client
- ▶ **exemplu:** <TCP,192.168.1.6:4444,193.226.51.6:80>

World Wide Web (`www`)

- ▶ creat la CERN și publicat în 1991
- ▶ resurse identificate prin Uniform Resource Locators (URL)
- ▶ pagini web prezentate prin hypertext
- ▶ HyperText Transfer Protocol (HTTP) RFC2616
- ▶ servesc pagini web pe portul 80
- ▶ acces encriptat prin HTTPS pe port 443 (curând implicit)
- ▶ **exemple:**
 - ▶ `http://fmi.unibuc.ro:80,`
 - ▶ `http://fmi.unibuc.ro`
 - ▶ `http://fmi.unibuc.ro:443`
 - ▶ `https://fmi.unibuc.ro:443`
 - ▶ `https://fmi.unibuc.ro`
- ▶ browser Unix: `lynx(1)`, `w3m(1)`
- ▶ crawler Unix: `curl(1)`, `wget(1)`

Acces la poștă

- ▶ Post Office Protocol (POP) RFC1939
- ▶ POP face o copie locală a mesajelor și le șterge de pe server
- ▶ Internet Message Access Protocol (IMAP) RFC3501
- ▶ IMAP operează asupra mesajelor direct pe server
- ▶ IMAP poate ține o copie (parțială) local
- ▶ porturi: POP 110, POP3S 995, IMAP 143, IMAPS 993
- ▶ mesageria ținută în format mbox sau maildir
- ▶ mbox: ține toate mesajele într-un singur fișier
- ▶ maildir: ține fiecare mesaj într-un fișier separat

Expediere mesaje

- ▶ Simple Message Transfer Protocol (SMTP) RFC821
- ▶ SMTP trimite un mesaj către un destinatar
- ▶ protocol vechi, nu cere exepeditor → spam
- ▶ protocol vechi, nu cere autentificare → spam
- ▶ porturi: SMTP 25, SMTPS
- ▶ soluții anti-spam
 - ▶ reverse DNS: legătură IP → nume
 - ▶ autentificare: utilizator și parolă, criptare
 - ▶ Sender Policy Framework (SPF): anti-email spoofing
 - ▶ baze de date cu IP-uri de spameri

File Transfer Protocol (ftp)

Transfer de fișiere

- ▶ protocol vechi din 1971, activ și astăzi RFC959
- ▶ autentificare și criptare FTPS
- ▶ port separat de comandă (21) și date (20)
- ▶ **exemplu:**
`ftp://[user[:password]@]host[:port]/url-path`
- ▶ comandă Unix: `ftp(1)`
- ▶ **exemple:**
 - ▶ `$ ftp alex@fmi.unibuc.ro`
 - ▶ `$ ftp -a fmi.unibuc.ro`
 - ▶ `$ ftp fmi.unibuc.ro -P 2121`

Tranfer de fișiere distribuit fără server

- ▶ comunicație numită peer-to-peer
- ▶ toți clienții transmit și primesc date
- ▶ cei ce doar transmit se numesc *seeds*
- ▶ cei ce primesc se numesc *leechers*
- ▶ mod eficient de transfer pentru date mari
- ▶ elimină presiunea de pe un singur server
- ▶ cel ce inițiază crează un fișier mic *.torrent* în care există informațiile legate de date și locul de unde se pot accesa

Secure Shell (ssh)

Protocol sigur de conectare la alt calculator

- ▶ specificat în RFC4253, port 22
- ▶ implementarea ubicuă OpenSSH creată de grupul OpenBSD
- ▶ <http://www.openssh.com>
- ▶ comandă Unix: `ssh(1)`
- ▶ **exemplu:** `$ ssh [user[:password]@]host[:port]`
- ▶ autentificare parolă, cheie asimetrică etc.
- ▶ o dată autentificat se deschide un terminal pe acel *host* în care puteți începe să dați comenzi
- ▶ se poate folosi interfața grafică cu argumentul `-X`
- ▶ folosit peste tot în programare și administrare

Secure Copy (scp)

Copiere prin protocolul SSH

- ▶ Secure Copy Protocol (SCP) nu există RFC
- ▶ funcționează similar cu comanda `cp(1)`
- ▶ are în plus prefixat *host-ul*
- ▶ comenzi Unix: `scp(1)`
- ▶ **exemple:**
 - ▶ `$ scp hello.c fmi.unibuc.ro:`
 - ▶ `$ scp hello.c alex@fmi.unibuc.ro:code/`
 - ▶ `$ scp -r project/ alex@fmi.unibuc.ro:`

SSH FTP (`sftp`)

Funcționalitate FTP prin protocolul SSH

- ▶ SSH File Transfer Protocol (SFTP) nu există RFC
- ▶ funcționează similar cu comanda `ftp(1)`
- ▶ profită de autentificare și criptografia de desubt (SSH)
- ▶ de obicei tot pe port 22 este servit
- ▶ comenzile protocolului SSH diferențiază între tipuri de sesiuni
- ▶ comenzi Unix: `sftp(1)`
- ▶ **exemplu:** `$ sftp alex@fmi.unibuc.ro`

Virtual Network Computing (vnc)

- ▶ bazat pe protocolul Remote Framebuffer (RFB) RFC6143
- ▶ oferă acces la interfața grafică de la distanță
- ▶ funcționează cu Windows, macOS și X (sistemul grafic Unix)
- ▶ VNC este cea mai cunoscută aplicație ce implementează și folosește RFB
- ▶ clientul se conectează la server pe port 5900
- ▶ comenzi Unix: `vncviewer(1)`, `vncserver(1)`
- ▶ **exemplu:**
 - ▶ `$ vncserver :0` – pornește server pe primul display
 - ▶ `$ vncviewer fmi.unibuc.ro`
 - ▶ `$ vncviewer fmi.unibuc.ro:2` – conectează la display-ul 3
 - ▶ `$ vncserver -kill :0` – oprește server

Remote Desktop (rdp)

- ▶ Remote Desktop Protocol, proprietar Windows
- ▶ funcționează similar cu VNC
- ▶ oferă în plus acces la sistemul audio și imprimantă
- ▶ clientul se conectează la server pe port 3389
- ▶ pe Windows clasic e permis un singur utilizator conectat
- ▶ pe Windows server se pot conecta mai mulți deodată
- ▶ comenzi Unix: `rdesktop(1)`, `xfreerdp(1)`
- ▶ **exemplu:**
 - ▶ `$ rdesktop 192.168.1.6`
 - ▶ `$ rdesktop fmi.unibuc.ro`
 - ▶ `$ rdesktop -r sound:remote -g 90% 141.85.225.153`

Elemente de configurare rețea

Utilizarea Sistemelor de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică
Email: paul.irofti@fmi.unibuc.ro

Rețele private

- ▶ adrese IP accesibile doar în rețeaua internă
- ▶ *host*-urile nu oferă servicii → nu comunică direct cu exteriorul
- ▶ exemplu: calculatoarele personale, stații de lucru
- ▶ rețelele private comunică cu exteriorul prin unul sau mai multe IP-uri publice
- ▶ de obicei aceste IP-uri publice aparțin unui server sau router
- ▶ server-ul are cel puțin două adrese IP: una privată și una publică

De la	Până la
10.0.0.0	10.255.255.255
172.16.0.0	172.31.255.255
192.168.0.0	192.168.255.255

Tabela: IP-uri private

Mască rețea

- ▶ adresa IP este împărțită în două părți: rețeaua și *host*-ul
- ▶ cele două sunt separate cu ajutorul unei măști pe biți
- ▶ exemplu: IP 192.168.1.17 și masca 255.255.255.0

IP	11000000	10101000	00000001	00010001
mască	11111111	11111111	11111111	00000000
rețea	11000000	10101000	00000001	00000000
	192	168	1	0

- ▶ biții setați indică partea de rețea – bloc contiguu
- ▶ biții zero indică partea de *host* – bloc contiguu
- ▶ notație CIDR (Classless Inter-Domain Routing): IP/*prefix*
- ▶ *prefix* – numărul de biți setați în mască
- ▶ exemplu: IP 192.168.1.17/24, rețea 192.168.1.0/24

Difuzare (Broadcast address)

- ▶ adresă de difuzare: date primite și distribuite în toată rețeaua
- ▶ 255.255.255.255 – broadcast către toate adresele IP
- ▶ broadcast în rețeaua locală → limited broadcast
- ▶ adresa: prefixul rețelei combinat + biții setați în partea de *host*
- ▶ exemplu: IP 192.168.1.17 și masca 255.255.255.0

rețea	11000000	10101000	00000001	00000000
mască	11111111	11111111	11111111	00000000
<i>host</i>	11111111	11111111	11111111	xxxxxxxx
broadcast	11000000	10101000	00000001	11111111
	192	168	1	255

- ▶ adresa broadcast în format CIDR: 192.168.1.255/24

- ▶ MAC = Media Access Control
- ▶ adresă unică de 48-biți stabilită de producătorul plăcii de rețea
- ▶ se mai numește adresă fizică sau adresă hardware
- ▶ reprezentare hexazecimal cu octeți separați prin : sau -
- ▶ exemplu: `ca:fe:de:ad:be:ef` sau `ca-fe-de-ad-be-ef`
- ▶ primii 3 octeți identifică producătorul
- ▶ adresare neierarhică → căutare grea în rețea
- ▶ adesea folosită drept un al doilea mod de identificare în rețea
- ▶ inițial adresa se schimba greu (aproape imposibil), acum este destul de facil

Dispozitive de transmisie

- ▶ placă de rețea
 - ▶ alias: network card, network adapter, NIC (Network Interface Controller)
 - ▶ primește și trimite pachete pentru un *host*
 - ▶ un *host* poate avea mai multe plăci de rețea
- ▶ switch
 - ▶ alias: hub, MAC bridge
 - ▶ conectează mai multe *host*-uri în rețeaua locală
 - ▶ transmite pachete bazându-se pe adresa MAC
 - ▶ nu are o adresă IP și nu participă altfel în rețea
- ▶ router
 - ▶ conectează mai multe calculatoare sau rețele
 - ▶ transmite pachete bazându-se pe adresa IP
 - ▶ conține tabele cu rute în rețea
 - ▶ citește destinația finală și folosește tabela cu rute pentru a trimite pachetul mai departe

Interfață de rețea

- ▶ alias: network interface
- ▶ de obicei e vorba despre o placă de rețea
- ▶ mai poate fi: un port hardware, un port în rețea, un socket
- ▶ in sistemul de operare configurăm interfața de rețea
- ▶ Linux: eth0, eth1, eth2, ... – *PlacăNumăr*
- ▶ Unix: em0, iwm0, axe0 – *ProducătorNumăr*
- ▶ comandă: `ifconfig(8)` – setează IP, mască, MAC etc.

Transmisie (routing)

- ▶ transmisia este făcută prin tabela cu rute
- ▶ alias: routing table, routing information base (RIB)
- ▶ conține rute către anumite rețele
- ▶ ponderea fiecărei căi (metrics)
- ▶ rutele reprezintă (parte din) topologia rețelei
- ▶ rute dinamice și statice
- ▶ *host* transmite un pachet către o adresă IP fără să îi pese de destinație către *gateway*
- ▶ *gateway*-ul se ocupă să trimită pe calea corectă folosind tabela de rute
- ▶ comandă: `route(8)`
- ▶ afișare rute IPv4: `$ route show -inet`

Exemplu: rute IPv4

```
$ route -n show -inet  
Routing tables
```

Internet:

Destination	Gateway	Iface
default	192.168.1.1	iwm0
127/8	127.0.0.1	lo0
127.0.0.1	127.0.0.1	lo0
192.168.1/24	192.168.1.6	iwm0
192.168.1.1	10:c3:7b:54:4f:98	iwm0
192.168.1.6	dc:53:60:98:c9:9d	iwm0
192.168.1.255	192.168.1.6	iwm0

Exemplu: rute IPv4+DNS

```
$ route show -inet  
Routing tables
```

```
Internet:
```

Destination	Gateway	Iface
default	router.asus.com	iwm0
loopback	localhost	lo0
localhost	localhost	lo0
192.168.1/24	sci	iwm0
router.asus.com	10:c3:7b:54:4f:98	iwm0
sci	dc:53:60:98:c9:9d	iwm0
192.168.1.255	sci	iwm0

- ▶ rețeaua privată este conectată la alte rețele printr-un *gateway*
- ▶ direcționarea pachetelor este făcută prin *gateway*-ul *default* (implicit)
- ▶ *gateway*-ul este la bază un router
- ▶ conține cel puțin două interfețe de rețea: IP public și local
- ▶ în tabela de rute are informații despre interior și exterior
- ▶ de obicei nu trimite pachetul la destinația finală ci la următorul router (denumit și *hop*)
- ▶ următorul router trimite mai departe pachetul către următorul *hop* sau către destinația finală
- ▶ comandă: `tracert(8)`

Exemplu: itinerariu pachete

```
$ traceroute fmi.unibuc.ro
traceroute to fmi.unibuc.ro (193.226.51.6), 64 hops max
 1  192.168.43.1 (192.168.43.1)
 2  * * *
 3  10.8.5.113 (10.8.5.113)
 4  10.8.5.129 (10.8.5.129)
 5  * * *
 6  static-10-220-128-98.rdsnet.ro (10.220.128.98)
 7  br01.bucuresti.rdsnet.ro (213.154.124.7)
 8  roedunet.bucuresti.rdsnet.ro (81.196.1.206)
 9  te-0-0-0-0.core1.nat.roedu.net (37.128.239.13)
10  37.128.232.30 (37.128.232.30)
11  37.128.230.74 (37.128.230.74)
12  fmi.unibuc.ro (193.226.51.6)
13  fmi.unibuc.ro (193.226.51.6)
14  fmi.unibuc.ro (193.226.51.6)
15  fmi.unibuc.ro (193.226.51.6)
16  ns1.fmi.unibuc.ro (193.226.51.1)
```

Network Address Translation (NAT)

- ▶ transformă o adresă din spațiul IP al unei rețele în spațiul altei rețele
- ▶ exemplu: $5.2.3.1/24 \leftrightarrow 192.168.1.1/24$
- ▶ util când se face legătura dintre o rețea privată și internet
- ▶ operație efectuată de router
- ▶ Fie o rețea privată unde *host*-urile au IP-uri private
 - ▶ un pachet pleacă de pe un *host* către un server din internet
 - ▶ ieșire: destinație IP global, sursă IP local
 - ▶ o dată trimis, așteaptă un răspuns
 - ▶ intrare destinație IP global, sursă IP global – cum ajunge pe *host*?

Configurarea unei mașini

Minim necesar pentru acces în rețea

- ▶ adresa IP a mașinii
- ▶ masca rețelei locale
- ▶ adresa IP gateway
- ▶ două adrese IP pentru servere DNS
- ▶ în situații limită se poate folosi 8.8.8.8 – DNS Google
- ▶ dacă aceste date sunt transmise static utilizatorului pot apărea probleme
 - ▶ la schimbarea IP-urilor rețelei 192.168.1.1/24 → 10.0.0.1/24
 - ▶ la schimbarea serverelor DNS
 - ▶ eroare umană: același IP este dat la doi utilizatori
 - ▶ lipsă adrese disponibile pentru utilizatori noi
 - ▶ când știu că un IP nu mai este folosit?

Dynamic Host Configuration Protocol (DHCP)

- ▶ este suficient conectarea cablului de rețea sau selectarea rețelei wireless
- ▶ informațiile de configurare sunt luate automat de pe server-ul DHCP
- ▶ alocare IP-uri dintr-un interval – alocare dinamică
- ▶ alocare specifică pentru un host – bazat pe adresa MAC

```
subnet 192.168.1.0 netmask 255.255.255.0 {  
    option routers 192.168.1.1;  
    range 192.168.1.100 192.168.1.200;  
    host sci {  
        hardware ethernet dc:53:60:98:c9:9d;  
        fixed-address 192.168.1.12;  
        option host-name "sci";  
    }  
}
```

Filtrarea pachetelor

Firewall-ul Linux este configurat cu ajutorul tabelelor de intrare și ieșire setate de utilizator cu ajutorul iptables(8)

- ▶ tabele (*chains*): INPUT, OUTPUT, FORWARD
- ▶ INPUT: pachetele care intră în rețea
- ▶ OUTPUT: pachetele care ies din rețea
- ▶ FORWARD: pachetele care trebuie trimise mai departe, nu sunt pentru rețeaua router-ului
- ▶ când un pachet trece printr-o tabelă îi putem aplica reguli
- ▶ cele mai comune reguli: acceptare sau respingere a pachetului
- ▶ este aplicată tot timpul prima regulă care se potrivește
- ▶ exemplu:
 - r1) permite trafic pe port 80
 - r2) blochează tot traficul
 - traficul pe port 80 este permis, restul este blocat

Parametrii

- ▶ -A – specifică numele tabelii (ex. -A OUTPUT)
- ▶ -p – protocol (ex. -p tcp)
- ▶ -dport – port destinație (ex. -dport 80)
- ▶ -j – verdict, *jump chain* (ex. -j DROP)
- ▶ exemple:

```
sudo iptables -A INPUT -p tcp -dport ssh -j ACCEPT
```

```
sudo iptables -A INPUT -j DROP
```
- ▶ listă comenzi existente: iptables -L
- ▶ salvează reguli: iptables-save > /etc/iptables.rules

Configurarea unui router

- ▶ configurare a cel puțin două interfețe: rețeaua locală și externă
- ▶ rețeaua locală este cea controlată de router
- ▶ router-ul decide ce spațiu de adrese alege devenind *gateway*
- ▶ dacă rețeaua locală este privată trebuie să facă NAT
- ▶ trebuie configurat ce port-uri sunt vizibile și cum este redirectionat accesul în rețeaua locală
- ▶ opțional: serviciu DNS local → acces rapid
- ▶ opțional: serviciu DHCP pentru rețeaua locală

Controlul versiunilor

Utilizarea Sistemelor de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică
Email: paul.irofti@fmi.unibuc.ro

Un scenariu des întâlnit

1. avem un program bun, stabil pe care vrem să-l schimbăm
2. schimbarea dorită se dovedește dificilă sau imposibilă
3. revinirea la vechiul program

Alternative punctul 2

- ▶ schimbarea e bună, dorim ambele versiuni ale programului
- ▶ schimbarea se poate aplica și altor programe existente

În orice caz este evident că trebuie să salvăm una sau mai multe copii ale programului de-a lungul evoluției sale.

Hello, World!

```
#include <stdio.h>
int main()
{
    printf("Hello , World!\n");
    return 0;
}
```

Vrem să adăugăm funcționalitate nouă: salută o persoană!

Hello, Alex!

```
#include <stdio.h>
int main(int argc, char *argv[])
{
    if (argc != 2) {
        printf("USAGE: %s name\n", argv[0]);
        return -1;
    }
    printf("Hello , %s!\n", argv[1]);
    return 0;
}
```

Compară fișiere și directoare

- ▶ `diff -u file1 file2` – arată diferențele între fișierul 1 și 2
- ▶ diferențele pot fi prezentate în mai multe feluri
- ▶ `-u` – modul uniform, cel mai des întâlnit
- ▶ fiecare linie începe fie cu un spațiu, un plus sau un minus
- ▶ `+` – linii adăugate
- ▶ `-` – linii înlăturate
- ▶ spațiu – linii ce conferă contextul schimbării
- ▶ dacă în fișier apar mai multe schimbări la distanțe mari (sute de linii) atunci fiecare schimbare este prezentată local
- ▶ începutul unei schimbări este marcat de două semne arond @@

Exemplu

```
$ diff -u hello.c.orig hello.c
--- hello.c.orig   Wed Apr 11 12:31:55 2018
+++ hello.c        Wed Apr 11 12:33:09 2018
@@ -1,6 +1,10 @@
 #include <stdio.h>
 -int main()
 +int main(int argc, char *argv[])
 {
 - printf("Hello , World!\n");
 + if (argc != 2) {
 +     printf("USAGE: %s name\n", argv[0]);
 +     return -1;
 + }
 + printf("Hello , %s!\n", argv[1]);
     return 0;
 }
```

patch(1)

Un patch este un fișier în care s-au salvat diferențele între versiunea veche și cea nouă a unui program.

Se generează cu ajutorul comenzii `diff(1)`

```
diff -u hello.c.orig hello.c > hello_name.patch
```

Se aplică cu ajutorul comenzii `patch(1)`

```
$ patch -p0 < hello_name.patch
```

```
Hmm... Looks like a unified diff to me...
```

```
The text leading up to this was:
```

```
|--- hello.c.orig  Wed Apr 11 12:31:55 2018  
|+++ hello.c       Wed Apr 11 12:33:09 2018
```

```
Patching file hello.c.orig using Plan A...
```

```
Hunk #1 succeeded at 1.
```

```
done
```

`-pNUM` specifică câte directoare din cale să fie eliminate

Revision Control în limbajul de specialitate

- ▶ gestionarea mai multor versiuni ale aceluiași fișier
- ▶ versiunile se mai numesc și revizii
- ▶ util în dezvoltarea programelor
- ▶ o revizie poate reprezenta
 - ▶ o versiune scoasă pe piață
 - ▶ rezolvarea unui defect
 - ▶ adăugarea unei noi funcționalități
- ▶ avantaje:
 - ▶ pași înainte și înapoi prin revizii
 - ▶ metoda biseției pentru a descoperi când a apărut un defect
 - ▶ motivarea existenței unui fragment de cod
 - ▶ lucrul în echipă
- ▶ dezavantaje:
 - ▶ necesită meticulozitate și organizare a muncii
 - ▶ lucru în plus față de programarea efectivă
 - ▶ complexitate sporită

- ▶ conține toate versiunile fișierelor și directoarelor unui proiect
- ▶ de obicei centralizat pe un server
- ▶ mod de lucru utilizatori (clienți)
 1. conectare și copie locală a *repository*-ului
 2. modifică unul sau mai multe fișiere
 3. transmiterea modificărilor serverului
 4. serverul crează o versiune nouă a proiectului

- ▶ copie locală a unui repository
- ▶ se mai numește și *working-copy*
- ▶ crează pe disk ultima versiune a tuturor fișierelor din repository
- ▶ poate conține și o bază de date locală cu versiunile anterioare
- ▶ o versiune este memorată de obicei prin diferențele față de cea anterioară

- ▶ înregistrează schimbările locale în *repository*
- ▶ conține și o descriere a modificărilor făcute
- ▶ este foarte important să detaliați cât mai mult
- ▶ ajută oamenii cu care colaborați și vă ajută și pe voi din viitor
- ▶ formatul des întâlnit este subiect urmat de o descriere de un paragraf

Example

[BUGFIX] Avoid NULL dereference during Emulator destruction.

Found during MemoryUnit!pmap_remove while the Emulator dtor has already executed and the MemoryUnit dtor was in progress.

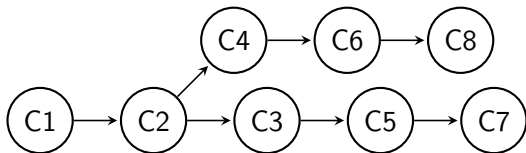
Sample:

tests/libs/pinky/win32api/ntdll/test_RtlGetNtVersionNumbers.exe

- ▶ actualizarea copiei locale
- ▶ aduce versiunile noi din repository în copia locală
- ▶ dacă există modificări locale pot apărea conflicte
- ▶ exemplu: doi programatori au modificat aceeași linie dintr-un fișier
- ▶ conflictele se rezolvă întâi automat
- ▶ dacă procesul eșuează este necesară intervenția utilizatorului
- ▶ pot apărea situații când există versiuni locale și versiuni noi în repository, atunci procesul este mai complex (vezi *merge*)

Branch

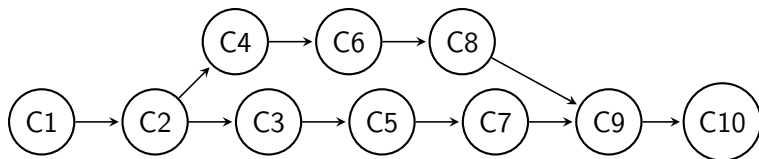
- ▶ liniar: fiecare commit depinde de cel precedent
- ▶ fie înlănțuirea $C1 \rightarrow C2 \rightarrow C3$
- ▶ un branch apare când cineva porește de la C2 și face un commit diferit C4 în continuarea lui C2 nu a lui C3



- ▶ branch, bifurcație, ramură
- ▶ ramura principală se mai numește *trunk*

Merge

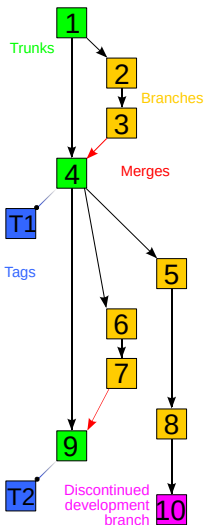
- ▶ operația *merge* integrează două sau mai multe schimbări
- ▶ acestea pot fi conflictuale sau nu



- ▶ dacă există conflict între cele două ramuri, C9 este o versiunea ce integrează branch-ul C4 și rezolvă conflictele
- ▶ dacă nu există nici un conflict, C9 este pur și simplu o modificare nouă

- ▶ *tag*: etichetă aplicată unui anumit *commit*
- ▶ folosită pentru a marca o versiune a produsului
- ▶ nomenclatură: alpha, beta, release candidate, release
- ▶ alpha – prima etapă înainte de lansarea unui produs, cerc redus (intern) de testare
- ▶ beta – a doua etapă îmbunătățită, cerc extins (extern) de testare
- ▶ release candidate (rc) – candidat pentru release
- ▶ rc1, rc2 – echivalenți alpha, beta
- ▶ release – lansarea produsului
- ▶ versiunea x.y.z – versiune majoră, minoră, patch (ex. 6.3.12)
- ▶ exemple *tag*: v1.1, v2.0alpha, v4.2beta, v5.6rc3

Exemplu istoric proiect



https://en.wikipedia.org/wiki/Version_control

- ▶ un singur *repository* pe un server
- ▶ fiecare utilizator trebuie să aibă acces
- ▶ versiunile anterioare și istoricul sunt păstrate pe server
- ▶ local copiem ultima versiune
- ▶ este necesar să fim conectați la server pentru a face un *commit*
- ▶ exemple: `cvs(1)`, `svn(1)`

- ▶ fiecare utilizator are *repository* propriu
- ▶ versiunile anterioare și istoricul sunt păstrate local
- ▶ nu este necesară rețeaua
- ▶ *commit*-urile se fac local
- ▶ sincronizare în rețea prin operații de *push* și *pull* între utilizatori
- ▶ *push* – trimite modificările către un alt *repository* (similar unui șir de operații *commit*)
- ▶ *pull* – preia modificările de la un alt *repository* (similar operației de *update*)
- ▶ cele două operații implică un *merge* și pot necesita intervenții suplimentare din partea utilizatorului pentru a rezolva conflicte
- ▶ tendință de centralizare *de-facto*
- ▶ exemple: `git(1)`, `hg(1)`

Servere web

Utilizarea Sistemelor de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică
Email: paul.irofti@fmi.unibuc.ro

Serverul Web

- ▶ comunică prin protocolul HyperText Transfer Protocol (HTTP)
- ▶ reprezintă fundația World Wide Web (WWW)
- ▶ servește pagini web, documente și alte resurse clienților
- ▶ paginile web: scrise cu HyperText Markup Language (HTML)
- ▶ navigare prin intermediul hyperlink-urilor din paginile web
- ▶ un hyperlink poate indica către o pagină a serverului web actual sau al altuia
- ▶ adresele urmează formatul Uniform Resource Locator (URL)
- ▶ `http://www.example.com/index.html`
 - ▶ `http` – protocolul
 - ▶ `www` – subdomeniu
 - ▶ `example.com` – domeniu, host, server
 - ▶ `.com` – top level domain (TLD)
 - ▶ `index.html` – fișierul ce conține pagina web

- ▶ funcționează pe modelul cerere-răspuns
- ▶ exemplu: clientul cere o pagină web, iar server-ul întoarce pagina în format HTML
- ▶ comunicația presupune existența protocolului IP pentru adresare și TCP pentru transportul datelor între server și client
- ▶ porturile implicite sunt: 80 și 443 pentru comunicare encriptată
- ▶ HTTP/1.0: o singură cerere-răspuns per conexiune
- ▶ HTTP/1.1: conexiunea stabilește o sesiune cu oricâte cereri-răspuns
- ▶ oferă autentificare de tip utilizator-parolă
- ▶ clientul mai este denumit și identificat ca *User Agent*

Cereri HTTP

- ▶ OPTIONS – afișează cererile HTTP acceptate de server
- ▶ GET – cere o resursă: pagină web, document etc.
- ▶ HEAD – la fel ca GET dar fără a prelua resursa, doar datele despre ea
- ▶ PUT – transmite date noi și le stochează la URL cerut
- ▶ POST – transmite date noi cu care inițializează o acțiune pe server
- ▶ DELETE – șterge resursa de la URL-ul transmis
- ▶ PATCH – aplică modificări resursei de la URL-ul transmis
- ▶ CONNECT – crează un tunel TCP/IP; pachete encriptate prin conexiune HTTP
- ▶ TRACE – funcție de tip ecou folosită la depanare; în general blocată

GET

- ▶ obține paginile web pentru *user agent*
- ▶ format simplu: cale către fișier și numele host-ului

```
GET /index.html HTTP/1.1  
host: fmi.unibuc.ro
```

```
HTTP/1.1 200 OK  
Content-Type: text/html  
Accept-Ranges: bytes  
ETag: "2065617787"  
Last-Modified: Mon, 25 May 2015 15:34:17 GMT  
Content-Length: 5940  
Date: Thu, 19 Apr 2018 09:35:53 GMT  
Server: lighttpd/1.4.26
```

```
[content]
```

PUT

- ▶ acceptă datele primite și le stochează la URL-ul cerut
- ▶ funcționalitate de tip upload ftp sau sftp
- ▶ de oricâte ori este apelat PUT rezultatul este același
- ▶ Content-type – tipul conținutului; trebuie înțeles de server
- ▶ Content-length – mărimea conținutului în bytes

```
PUT /new.html HTTP/1.1
Host: example.com
Content-type: text/html
Content-length: 16
```

```
<p>New File </p>
```

POST

- ▶ transmite date noi și inițializează o acțiune pe server
- ▶ acțiunea depinde de server și conținut
- ▶ poate fi redusă la o operațiune tip PUT
- ▶ poate crea o resursă la altă adresă
- ▶ apelat repetat, poate avea efecte diferite asupra conținutului serverului
- ▶ este folosit pentru a scrie pe forumuri

POST / HTTP/1.1

Host: example.com

Content-Type: application/x-www-form-urlencoded

Content-Length: 21

subj=Hello&msg=World

Coduri de eroare

- ▶ prima linie din răspuns la cerere; format din 3 cifre
- ▶ exemplu: HTTP/1.1 200 OK
- ▶ anunță ce tip de răspuns urmează
- ▶ 1xx – mesaj informativ
- ▶ 2xx – operația a decurs cu succes
- ▶ 3xx – redirectare (ex. pagina a fost mutată în alt director)
- ▶ 4xx – eroare client (ex. pagina cerută nu există)
- ▶ 5xx – eroare server (ex. cererea a expus un defect în server)
- ▶ coduri des întâlnite
 - ▶ 404 – pagina nu a fost găsită
 - ▶ 403 – acces interzis
 - ▶ 502 – server-ul funcționa ca gateway și nu s-a putut comunica cu un server extern
 - ▶ 301 – site-ul a fost mutat permanent la adresa cuprinsă în răspuns

Software care acționează în numele utilizatorului

- ▶ implicit reprezintă browser-ul și conține date legate de el
- ▶ exemplu date: nume, versiune, sistem de operare etc.
- ▶ Firefox: Mozilla/[version] ([system and browser information]) [platform] ([platform details]) [extensions]
- ▶ concret: Mozilla/5.0 (X11; OpenBSD amd64; rv:59.0) Gecko/20100101 Firefox/59.0
- ▶ poate fi și un automat: robot, crawler, bot
- ▶ agenții automați trebuie să respecte regulile din fișierul robots.txt

- ▶ dictează ce directoare și fișiere pot fi accesate de automate
- ▶ servește un singur (sub)domeniu
- ▶ exemplu: separat pentru `fmi.unibuc.ro` și `unibuc.ro`
- ▶ directive
 - ▶ User-Agent – specifică pentru ce agenți se aplică regulile
 - ▶ Disallow – căi și fișiere interzise
 - ▶ Allow – căi permise (ex. interzic un director întreg, dar permit un fișier anume din acel director)
 - ▶ Crawl-delay – perioada minimă în secunde între vizite
 - ▶ Host – specifică ce *mirror* să folosească pentru acces

Exemplu `http://fmi.unibuc.ro/robots.txt`

```
User-agent: *  
Disallow: /devel  
Disallow: /stest  
Disallow: /vechi  
Disallow: /rectorat  
Disallow: /phpmyadmin
```

- ▶ limitarea accesului la anumite fișiere sau directoare
- ▶ permisiune prin autentificare cu utilizator și parolă
- ▶ metode interne incluse în protocolul HTTP
 - ▶ basic access
 - ▶ digest access
- ▶ există metode externe mai bune, puternice criptografic
 - ▶ autentificare prin cheie publică
 - ▶ kerberos: autentificare reciprocă client-server
 - ▶ cookie sessions: identificarea comunicației printr-o singură autentificare la început

- ▶ cea mai simplă formă de autentificare
- ▶ datele sunt trimise în clar în cerere
- ▶ nu folosește criptografie, cookie-uri sau alte mecanisme
- ▶ folosit împreună cu HTTPS pentru securitate

- ▶ autentificare `username:realm:password`
- ▶ realm: setul de pagini care folosesc același utilizator și parolă
- ▶ funcție de hashing (MD5) pentru a transmite acreditările
- ▶ poate fi folosit cu HTTP, deși nu este cea mai sigură metodă
- ▶ fișierul `.htdigest` conține datele generate cu `htdigest(1)`:
`alex:fmi:5ea41921c65387d904834f8403185412`

- ▶ comunicația HTTP nu reține contextul dialogului (*stateless*)
- ▶ de multe ori avem nevoie să reținem acțiuni trecute (*stateful*)
- ▶ cookie-urile sunt fișiere text mici pe care le reține browser-ul
- ▶ exemple: date de autentificare, istoric, ce buton a fost apăsat
- ▶ în funcție de scop, există mai multe tipuri
 - ▶ session – ținute în memorie temporară, expiră după ce utilizatorul părăsește site-ul
 - ▶ persistent – este salvată pe disk cu o dată de expirare
 - ▶ advertisement – folosite pentru reclame, conțin istoricul, sunt de tip persistent
- ▶ conține un nume, o valoare și zero sau mai multe atribute
- ▶ atributele au o structură cheie-valoare
- ▶ construcție
 - ▶ server: emite `Set-Cookie:` cu nume, valoare
 - ▶ client: la următoarele cereri pune cookie-urile în `Cookie:`

Cookie: exemplu comunicare

Clientul cere o pagină

```
GET /index.html HTTP/1.1  
Host: fmi.unibuc.ro
```

Serverul răspunde și setează două cookie-uri

```
HTTP/1.0 200 OK  
Content-type: text/html  
Set-Cookie: lang=ro  
Set-Cookie: sessionToken=deadbeef;  
Expires=Fri, 03 Jun 2018 12:00:00 GMT+2
```

Clientul cere o altă pagină și anunță ce cookie-uri folosește

```
GET /examene.html HTTP/1.1  
Host: fmi.unibuc.ro  
Cookie: lang=ro; sessionToken=deadbeef
```

HTTPS prin TLS

- ▶ comunicația HTTP se face în mod normal prin text
- ▶ oricine din rețea poate vedea traficul
- ▶ soluție: transmiterea pachetelor HTTP criptat
- ▶ protocolul HTTP rămâne același
- ▶ peste este adăugat protocolul Transport Layer Security (TLS)
- ▶ TLS este proiectat exact pentru astfel de operați
- ▶ alte exemple de protocol necriptat învelit în TLS: FTP, SMTP, IMAP
- ▶ în URL protocolul devine `https` și portul implicit 443
- ▶ criptografia este bazată pe chei publice certificate de o autoritate
- ▶ certificarea garantează identitatea posesorului cheii private

Un certificat conține

- ▶ informații legate de cheia asimetrică
- ▶ informații legate de identitatea posesorului cheii private
- ▶ posesorul mai este denumit și subiect (*subject*)
- ▶ semnătura digitală a unui terț care a verificat conținutul certificatului
- ▶ acest terț mai este numit emițător (*issuer*)

Cheia publică va fi folosită de client dacă

- ▶ semnătura este validă
- ▶ programul care verifică certificatul are încredere în emițător
- ▶ rezultă un lanț al slăbiciunilor (*chain of trust*)

Din 2016 certificare gratuită prin Let's Encrypt!

- ▶ conținutul oferit de server este în format HTML
- ▶ la baza limbajului: `<tag>content</tag>`
- ▶ tag-ul decide cum este afișat conținutul
 - ▶ `<html>` – demarcă începutul și sfârșitul documentului
 - ▶ `<head>` – conține descrierea paginii
 - ▶ `<title>` – titlul paginii
 - ▶ `<meta>` – cuvinte cheie folosite la căutare
 - ▶ `<base>` – calea absolută; baza link-urilor relative din pagină
 - ▶ `<body>` – demarcă începutul și sfârșitul documentului
 - ▶ `<header>`, `<h1-6>` – titlu de mare (1) la mic (6)
 - ▶ `<div>`, `<p>`, `<br>` – secțiune, paragraf, linie nouă
 - ▶ `<b>`, `<i>`, `<u>` – bold, italic, underline
 - ▶ `<a>`, `<img>` – hyperlink, imagine
- ▶ tag-urile pot avea și attribute stabilite la început
 - ▶ `<div style="background-color:lightblue">...</div>`
 - ▶ `<a href="experience.html">Experience</a>`
 - ▶ ``

Common Gateway Interface (CGI)

- ▶ protocol prin care serverele web execută programe normale
- ▶ scopul acestor programe este de a genera pagini HTML
- ▶ implementarea serverului decide modul de execuție
- ▶ programele primesc o formă brută HTML prin variabile de mediu (ca \$PATH)
- ▶ majoritatea variabilelor de mediu sunt prefixate cu \$HTML_
- ▶ acestea conțin datele de intrare pentru program
- ▶ în general folosit în câmpuri <form>: butonul de *submit* după ce a fost tastat un mesaj pe forum
- ▶ când butonul este apăsat, conținutul este trimis unui program care execută mai multe operații
 - ▶ validează datele de intrare
 - ▶ înlocuiește formule speciale în text cu imagini sau cod HTML
 - ▶ adaugă mesajul în baza de date și actualizează profilul utilizatorului care a făcut operația

Limbaje ce folosesc protocolul CGI

- ▶ programele se pun de obicei în directorul `cgi-bin`
- ▶ istoric: programe specializate pentru site scrise în C
- ▶ limbajele de scripting existente profită și oferă programe generice CGI care oferă funcționalitate completă
- ▶ modul de funcționare a unui program CGI pentru un limbaj
 1. programul CGI primește datele de intrare
 2. datele sunt transformate într-o formă acceptată de compilator
 3. datele sunt trimise printr-un socket compilatorului
 4. procesarea și trimiterea datelor rezultate de către compilator
 5. aplică încă o operație de transformare a datelor din formatul primit de la compilator în cod HTML
 6. trimite serverului pagina web
 7. serverul trimite pagina clientului
- ▶ exemple: Perl, Ruby, Python

Baze de date

Utilizarea Sistemelor de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică
Email: paul.irofti@fmi.unibuc.ro

- ▶ adesea programele conțin algoritmi care procesează date
- ▶ modul în care sunt citite datele de intrare și scrise datele de ieșire ale unui algoritm au un impact direct asupra timpului de execuție
- ▶ date de intrare
 - ▶ trebuie găsite pe disc
 - ▶ aduse în memorie
 - ▶ parcurse de program
 - ▶ traduse în structurile proprii de date
 - ▶ trimise mai departe algoritmului
- ▶ date de ieșire
 - ▶ parcurse de program
 - ▶ traduse în formatul de pe disk
 - ▶ scrise pe disk
- ▶ avem nevoie de o structură eficientă a datelor

- ▶ frecvent datele au în spate o structură intrinsecă
- ▶ reprezentarea structurată aduce multe beneficii
- ▶ stocare – cu cât știm mai multe informații despre date, cu atât le putem împărți și organiza eficient pe disk
- ▶ parcurgere
 - ▶ extragem strict informația structurală care ne interesează
 - ▶ traversare rapidă
 - ▶ nu este necesar să încărcăm în memorie tot setul de date
- ▶ căutare eficientă
 - ▶ cu cât structura este mai laborioasă cu atât putem căuta mai eficient după informații specifice
 - ▶ citim minimul de informație necesară verdictului: găsit sau nu
 - ▶ exemple tehnici: indexare, caching, hashing
- ▶ reprezentare – generare de rapoarte, grafice, tabele care prezintă o idee, o tendință, pe care vrem să o evidențiem

Modelul entitate-asociere

- ▶ o entitate este o chestie care poate fi identificată în mod unic
- ▶ exemple: persoană, casă, mașină, artist
- ▶ o entitate are unul sau mai multe attribute
- ▶ exemplu: persoană – sex, vârstă, educație, ocupație
- ▶ o entitate are una sau mai multe relații cu alte entități
- ▶ exemplu: o persoană are doi părinți, patru bunici, câțiva copii
- ▶ o relație poate avea de asemenea una sau mai multe attribute
- ▶ exemplu: o relație de prietenie are o dată de început și una de sfârșit
- ▶ gramatical entitatea este substantivul, relația verbul și atributul adjectivul sau adverbul

Exemplu: Sistemul de coordonate carteziane

- ▶ entitatea este punctul
- ▶ attributele sale sunt coordonatele în fiecare dimensiune
- ▶ relațiile între puncte formează formele geometrice

- ▶ modul cel mai des întâlnit de structurare a datelor: linii și coloane
- ▶ fiecare tabelă reprezintă o entitate (ex. studenți)
- ▶ fiecare linie reprezintă o intrare, un element de acel tip (ex. studentul Alex)
- ▶ fiecare coloană reprezintă un atribut al entității (ex. nume, grupă, serie, note)
- ▶ relațiile reprezentate cu ajutorul coloanelor: o coloană poate indica către o altă intrare dintr-un tabel

Exemplu tabele: relație artist-melodie

ID	Artist	ID Melodie
1	Filarmonica George Enescu	512
13	Metallica	365
42	Mile Davis	88

ID	Melodie
88	So What
365	Frantic
512	Mahler 4

Modelul cheie-valoare

- ▶ nu există entități
- ▶ doar chei (attribute) și valori
- ▶ exemple sintaxă: `cheie = valoare`, `cheie : valoare`
- ▶ folosit pentru fișiere de configurare (ex. `/etc/hosts`)

Format INI

Format fișier tip cheie-valoare

- ▶ proprietăți cheie=valoare
- ▶ opțional separate în secțiuni [section-name]
- ▶ liniile comentate încep cu ;
- ▶ cele mai răspândite în Windows
- ▶ folosit de PHP (ex. `php.ini`)

```
; modificat la data de 01.05.2018
[student]
prenume=Alex
nume=Alexandrescu
grupa=151

[uso]
laborator=32
proiect=15
verificare=25
```

Comma-separated values

- ▶ stochează date tabulare în fișiere text
- ▶ fiecare linie din fișier reprezintă o intrare în tabel
- ▶ valoarea de pe fiecare coloană este separată prin virgulă
- ▶ toate intrările trebuie să aibă același număr de proprietăți

Tabela artist devine

```
1, Filarmonica George Enescu ,512  
13, Metallica ,365  
42, Mile Davis ,88
```

- ▶ format text de stocare a informațiilor
- ▶ urmează modelului entitate-asociere
- ▶ folosit pentru transportul datelor în rețea
- ▶ tag – desemnează începutul și sfârșitul unei entități
- ▶ element – conținutul, intrarea
- ▶ atribut – definește o proprietate a entității; se află în interiorul tag-ului
- ▶ formatul HTML este un subset al formatului XML

Exemplu XML

```
<artists>
  <artist>
    <id>1</id>
    <name>Filarmonica George Enescu</name>
    <melodie>512</melodie>
  </artist>
  <artist>
    <id>13</id>
    <name>Metallica </name>
    <melodie>365</melodie>
  </artist>
  <artist>
    <id>42</id>
    <name>Mile Davis</name>
    <melodie>88</melodie>
  </artist>
</artists>
```

- ▶ transformarea datelor într-un format de stocat pe disk
- ▶ formatul poate fi unul dintre cele descrise mai sus sau altele
- ▶ aspecte importante
 - ▶ portabilitate – formatul să fie standardizat pentru ca datele să fie accesate și de alte programe
 - ▶ stocare eficientă – alegerea unui format cât mai succint pentru a minimiza spațiul de disk
 - ▶ transport – format text sau binar în funcție de cum se transmit datele
- ▶ operația inversă se numește deserializare

Baze de date relaționale

- ▶ soluții software de stocare a datelor
- ▶ urmează modelul entitate-asociere
- ▶ rezolvă problema stocării: propriu format de fișier de stocare
- ▶ serializare și deserializare transparentă
- ▶ asigură integritatea stocării și manipulării datelor
- ▶ datele sunt organizate în tabele
- ▶ soluțiile folosesc un limbaj comun: Structured Query Language (SQL)
 - ▶ expresii – produc valori în tabele sau tabele
 - ▶ clauze – specifică condițiile în care să se aplice comanda
 - ▶ predicate – verifică valoarea de adevăr a unei expresii
 - ▶ interogare – extrage date în funcție de criteriile date

SQL: baze de date

- ▶ creare: `CREATE DATABASE DatabaseName;`
- ▶ afișare: `SHOW DATABASES;`
- ▶ selectare: `USE DatabaseName;`
- ▶ eliminare: `DROP DATABASE DatabaseName;`
- ▶ exemplu:

```
CREATE DATABASE AristMelodie;  
USE ArtistMelodie;
```

SQL: tabele

- ▶ creare:

```
CREATE TABLE TableName(  
    column1 datatype ,  
    column2 datatype ,  
    column3 datatype ,  
    .....  
    columnN datatype ,  
    PRIMARY KEY( one or more columns )  
);
```

- ▶ afișare: DESC TableName;
- ▶ eliminare: DROP TABLE TableName;
- ▶ exemplu:

```
CREATE TABLE artist(  
    ID int ,  
    artist varchar(255),  
    ID_melodie int ,  
    PRIMARY KEY(ID)  
);
```

SQL: Adăugare date

- ▶ comanda:

```
INSERT INTO TableName (column1 , column2 , ... columnN)  
VALUES (value1 , value2 , ... valueN );
```

- ▶ corespondență coloană-valoare
- ▶ nu trebuie specificate coloanele dacă adăugăm valori în toate
- ▶ ordinea este importantă în ambele cazuri
- ▶ exemplu:

```
INSERT INTO artist (ID , artist , ID_meldie)  
VALUES (1, Filarmonica George Enescu , 512);
```

SQL: interogare

- ▶ comanda:

```
SELECT column1 , column2 , columnN
FROM TableName
WHERE [condition];
```

- ▶ clauza WHERE este opțională
- ▶ exemplu: `SELECT ID,artist FROM artist;`

ID	NAME
1	Filarmonica George Enescu
13	Metallica
42	Mile Davis

- ▶ WHERE conține expresii logice care ajută filtrarea
 - ▶ `WHERE artist='Metallica'`
 - ▶ `WHERE ID < 10`
 - ▶ `WHERE artist='Metallica' AND ID_melodie='88'`

SQL: modificare date

- ▶ actualizare:

```
UPDATE TableName  
  SET column1 = value1 ,... columnN = valueN  
  WHERE [condition];
```

- ▶ eliminare:

```
DELETE FROM TableName  
  WHERE [condition];
```

SQL: utilizatori

- ▶ nou:

- ▶ mysql: `CREATE USER 'jeffrey'@'localhost' IDENTIFIED BY 'password';`
- ▶ psql: `$ createuser joe`
- ▶ sqlite: nu are utilizatori

- ▶ roluri:

```
CREATE [OR REPLACE] ROLE [IF NOT EXISTS] role
[WITH ADMIN
 {CURRENT_USER | CURRENT_ROLE | user | role}]
```

- ▶ acordarea drepturilor se face cu comanda GRANT; sintaxă specifică soluției software
- ▶ exemplu MySQL drepturi depline

```
GRANT ALL ON *.* TO 'admin'@'localhost'
IDENTIFIED BY 'password'
WITH GRANT OPTION;
```

- ▶ salvarea informațiilor dintr-o bază de date se face diferit în funcție de soluția software
- ▶ mysql: `$ mysqldump DatabaseName > dbname.sql`
- ▶ psql: `$ pg_dump DatabaseName > dbname.sql`
- ▶ sqlite:

```
sqlite> USE DatabaseName;  
sqlite> .output dbname.sql  
sqlite> .dump  
sqlite> .quit
```
- ▶ rezultatul final este un fișier text cu o înșiruire de comenzi SQL care crează baza de date, tabelele, și adaugă datele în tabele

Programare în Shell

Utilizarea Sistemelor de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică
Email: paul.irofti@fmi.unibuc.ro

- ▶ de multe ori vrem să avem o singură comandă pentru un șir de comenzi
- ▶ comenzile sunt preponderent comenzi shell sau din sistem, nu operații logice sau aritmetice
- ▶ avem nevoie să scriem un program scurt rapid
- ▶ vrem să funcționeze pe orice alt sistem pentru a repeta un set de operații de configurare sau administrare
- ▶ bazat pe CS2043 Unix and Scripting de la Cornell University (cursurile 9 și 10)

- ▶ `sh(1)` – Bourne shell
 - ▶ printre primele shelluri
 - ▶ disponibilă oriunde în `/bin/sh`
 - ▶ funcționalitate redusă
 - ▶ portabilitate
- ▶ `csh(1)` – C shell, comenzi apropiate de lucru în C
- ▶ `ksh(1)` – Korn shell, bazat pe `sh(1)`, succesor `csh(1)`
- ▶ `bash(1)` – Bourne Again shell
 - ▶ cea mai răspândită
 - ▶ există și în Windows 10
 - ▶ majoritatea scripturilor moderne scrise în `bash(1)`
 - ▶ acest rezultat a dus la o lipsă de portabilitate
 - ▶ proiect mare cu multe linii de cod și probleme de securitate

Variabile Shell

- ▶ două tipuri de variabile: locale și de mediu
- ▶ variabilele locale există doar în instanța curentă a shell-ului
- ▶ convenție: variabilele locale se notează cu litere mici
- ▶ asignare: `x=1` (fără spații!)
- ▶ `x = 1`: se interpretează drept comanda `x` cu argumentele `=` și `1`
- ▶ folosirea valorii unei variabile se folosește prin prefixarea numelui cu `$`
- ▶ conținutul oricărei variabile poate fi afișat cu comanda `echo(1)`

```
$ x=1
```

```
$ echo $x
```

```
1
```

Variabile de mediu

- ▶ folosite de sistem pentru a defini modul de funcționare a programelor
- ▶ shell-ul trimite variabilele de mediu proceselor sale copil
- ▶ `env(1)` – afișează toate variabilele de mediu setate
- ▶ variabile importante
 - ▶ `$HOME` – directorul în care se țin datele utilizatorului curent
 - ▶ `$PATH` – listă cu directoare în care se caută executabilele
 - ▶ `$SHELL` – programul de shell folosit implicit
 - ▶ `$EDITOR` – programul de editare fișiere implicit
 - ▶ `$LANG` – limba implicită (ex. `ro_RO.UTF-8`)
- ▶ `export(1)` – se folosește pentru a seta o variabilă de mediu
- ▶ `printenv(1)` – se folosește pentru a afișa o variabilă de mediu

```
$ export V=2
$ printenv V
2
$ echo $V
2
```

Exemplu: Variabilă locală

```
$ LANG=ro_RO.UTF-8
$ printenv LANG
$ echo $LANG
ro_RO.UTF-8
$ bash
bash-4.4$ printenv LANG
bash-4.4$ t
bash: t: command not found
bash-4.4$ echo $LANG

bash-4.4$ exit
```

Exemplu: Variabilă de mediu

```
$ export LANG=ro_RO.UTF-8
$ printenv LANG
ro_RO.UTF-8
$ bash
bash-4.4$ printenv LANG
ro_RO.UTF-8
bash-4.4$ echo $LANG
ro_RO.UTF-8
bash-4.4$ t
bash: t: comandă negăsită
```

Variable expansion

Variabilele pot fi mai mult decât simplii scalari

- ▶ `$(cmd)` – evaluează întâi comanda `cmd`, iar rezultatul devine valoarea variabilei

```
$ echo $(pwd)
/home/paul
$ x=$(find . -name \*.c)
$ echo $x
./batleft.c ./pcie.c ./maxint.c ./eisaid.c
```

- ▶ `$((expr))` – evaluează întâi expresia aritmetică, iar rezultatul devine valoarea variabilei

\$ x=1	
\$ echo \$((1+1))	\$ echo \$((x++))
2	1
\$ echo \$((x+1))	\$ echo \$((x++))
2	2
\$ echo \$((x<1))	\$ echo \$((++x))
0	4

Quoting

Șirurile de caractere sunt interpretate diferit în funcție de citare:

- ▶ Single quotes ''

- ▶ toate caracterele își păstrează valoarea
- ▶ caracterul ' nu poate apărea în șir, nici precedat de "\"
- ▶ exemplu:

```
$ echo 'Am variabila $x'  
Am variabila $x
```

- ▶ Double quotes ""

- ▶ caractere speciale \$ ' \ (opțional !)
- ▶ restul caracterelor își păstrează valoarea
- ▶ exemplu:

```
$ echo "$USER has home in $HOME"  
paul has home in /home/paul
```

- ▶ Back quotes `` – funcționează ca \$()

```
$ echo "Today is `date`"  
Today is Wed May  2 18:00:08 EEST 2018
```

Înlănțuirea comenzilor

- ▶ `cmd1; cmd2` – înlănțuire secvențială, `cmd2` imediat după `cmd1`
- ▶ `cmd1 | filtru | cmd2` – ieșirea comenzii din stânga este intrarea celei din dreapta operatorului `|`
- ▶ `cmd1 && cmd2` – execută a doua comandă doar dacă prima s-a executat cu succes
- ▶ `cmd1 || cmd2` – execută a doua comandă doar dacă prima a eșuat
- ▶ exemplu:

```
$ mkdir acte && mv *.docx acte/  
$ ls -lR | tee files.lst | wc -l  
$ ssh example.org || echo "Connection failed!"
```

Definition

Script-urile sunt programe scrise pentru un mediu run-time specific care automatizează execuția comenzilor ce ar putea fi executate alternativ manual de către un operator uman.

- ▶ nu necesită compilare
- ▶ execuția se face direct din codul sursă
- ▶ de acea programele ce execută scriptul se numesc interpretatoare înloc de compilatoare
- ▶ comenzile executate se mai numesc și byte-code
- ▶ astăzi granița dintre compilatoare și interpretatoare nu mai este atât de clară (JIT, tiered compilation)
- ▶ exemple: perl, ruby, python, sed, awk, ksh, csh, bash

Shebang (#!)

- ▶ reprezintă prima linie din orice script Unix
- ▶ are forma `#!/path/to/interpreter`
- ▶ exemple: `#!/bin/sh`, `#!/usr/bin/python`
- ▶ pentru portabilitate folosiți `env(1)` pentru a găsi unde este instalat interpretatorul
- ▶ exemple: `#!/usr/bin/env ruby`, `#!/usr/bin/env perl`
- ▶ oriunde altundeva în script `#` este interpretat ca început de comentariu și restul linei este ignorată (echivalent `//` în C)
- ▶ introdusă de Denis Ritchie circa 1979

Exemplu: hello.sh

1. Scriem fișierul `hello.sh` cu comenzile dorite

```
#!/bin/sh
```

```
# Salute the user  
echo "Hello , $USER!"
```

2. Permite executia: `chmod +x hello.sh`
3. Executăm:

```
$ ./hello.sh  
Hello , paul!
```

Variable speciale în scripturi

- ▶ `$1, $2, ..., ${10}, ${11}, ...` – argumentele în ordinea primită
- ▶ dacă numărul argumentului are mai mult de două cifre, trebuie pus între acolade
- ▶ `$0` – numele scriptului (ex. `hello.sh`)
- ▶ `$#` – numărul de argumente primite
- ▶ `$*` – toate argumentele scrise ca `"$1 $2 ... $n"`
- ▶ `@` – toate argumentele scrise ca `"$1" "$2" ... "$n"`
- ▶ `$?` – valoarea ieșirii ultimei comenzi executate
- ▶ `$$` – ID-ul procesului curent
- ▶ `$_` – ID-ul ultimului proces suspendat din execuție

Exemple: argumente script

- ▶ `add.sh` – adună două numere

```
#!/bin/sh  
echo $(( $1 + $2 ))
```

apel:

```
$ sh add.sh 1 2  
3
```

- ▶ `tolower.sh` – imită funcția din C `tolower(3)`

```
#!/bin/sh  
tr '[A-Z]' '[a-z]' < $1 > $2
```

apel:

```
$ echo "WHERE ARE YOU?" > screaming.txt  
$ ./tolower.sh screaming.txt decent.txt  
$ cat decent.txt  
where are you?
```

Pentru scripturi mai complexe avem nevoie, ca în orice limbaj, de blocuri de control

- ▶ condiționale – `if`, `test [ ]`, `case`
- ▶ iterative – `for`, `while`, `until`
- ▶ comparative – `-ne`, `-lt`, `-gt`
- ▶ funcții – `function`
- ▶ ieșiri – `break`, `continue`, `return`, `exit`

- ▶ cuvinte cheie: if, then, elif, else, fi
- ▶ exemplu ce folosește toate cuvintele cheie:

```
if    cmd1
then
    cmd2
    cmd3
elif  cmd4
then
    cmd5
    cmd6
else
    cmd7
fi
```

- ▶ o condiție este adevărată dacă comanda asociată este executată cu succes ($\$?=0$)

Exemplu: if

Caută în fișier date și le adaugă dacă nu le găsește

```
#!/bin/sh
if grep "$1" $2 > /dev/null
then
    echo "$1 found in file $2"
else
    echo "$1 not found in file $2, appending"
    echo "$1" >> $2
fi
```

apel:

```
$ ./text.sh who decent.txt
who not found in file decent.txt, appending
$ cat decent.txt
where are you?
who
```

- ▶ nu dorim să testăm tot timpul execuția unei comenzi
- ▶ există expresii pentru a compara sau verifica variabile
- ▶ `test expr` – evaluează valoarea de adevăr a lui `expr`
- ▶ `[ expr ]` – efectuează aceeași operație (**atenție la spații!**)
- ▶ expresiile pot fi legate logic prin
 - ▶ `[ expr1 -a expr2 ]` – și
 - ▶ `[ expr1 -o expr2 ]` – sau
 - ▶ `[ ! expr ]` – negație

Expresii test: numere

- ▶ [n1 -eq n2] - $n_1 = n_2$
- ▶ [n1 -ne n2] - $n_1 \neq n_2$
- ▶ [n1 -ge n2] - $n_1 \geq n_2$
- ▶ [n1 -gt n2] - $n_1 > n_2$
- ▶ [n1 -le n2] - $n_1 \leq n_2$
- ▶ [n1 -lt n2] - $n_1 < n_2$

Expresii test: șiruri de caractere

- ▶ `[ str ]` – str are lungime diferită de 0
- ▶ `[ -n str ]` – str nu e gol
- ▶ `[ -z str ]` – str e gol ("")
- ▶ `[ str1 = str2 ]` – stringuri identice
- ▶ `[ str1 == str2 ]` – stringuri identice
- ▶ `[ str1 != str2 ]` – stringuri diferite

Expresii test: fişiere

- ▶ `-e path` – verifică dacă există calea `path`
- ▶ `-f path` – verifică dacă `path` este un fişier
- ▶ `-d path` – verifică dacă `path` este un director
- ▶ `-r path` – verifică dacă aveţi permisiunea de a citi `path`
- ▶ `-w path` – verifică dacă aveţi permisiunea de a scrie `path`
- ▶ `-x path` – verifică dacă aveţi permisiunea de a executa `path`

while

- ▶ execută blocul cât timp comanda `cmd` se execută cu succes

```
while cmd
do
    cmd1
    cmd2
done
```

- ▶ în loc de comandă putem avea o expresie de test
- ▶ într-o singură linie: `while cmd; do cmd1 cmd2; done`

Exemplu: while

Afișează toate numerele de la 1 la 10:

```
i=1
while [ $i -le 10 ]
do
    echo "$i"
    i=$(( $i+1 ))
done
```

Sau într-o singură linie:

```
i=1; while [ $i -le 10 ]; do echo "$i"; i=$(( $i+1 ));
done
```


until

- ▶ execută blocul cât timp comanda `cmd` se execută **fără** succes

```
until cmd
do
    cmd1
    cmd2
done
```

- ▶ în loc de comandă putem avea o expresie de test
- ▶ într-o singură linie: `until cmd; do cmd1 cmd2; done`

for

- ▶ execută blocul pentru fiecare valoare din listă

```
for var in str1 str2 ... strN
do
    cmd1
    cmd2
done
```

- ▶ var ia pe rând valoarea str1, pe urmă str2 până la strN
- ▶ de regulă comenzile din bloc (cmd1, cmd2) sunt legate de var
- ▶ comanda for are mai multe forme, aceasta este cea mai întâlnită
- ▶ într-o singură linie:
for var in str1 str2 ... strN; do cmd1 cmd2; done

Exemplu: for

Compilează toate fişierele C din directorul curent

```
for f in *.c
do
    echo "$f"
    cc $f -o $f.out
done
```

Sau într-o singură linie:

```
for f in *.c; do echo "$f"; cc $f -o $f.out; done
```

for tradițional

- ▶ formă întâlnite doar în unele shell-uri, **nu este portabil**
- ▶ execută blocul urmând o sintaxă similară C

```
for (( i=1; i<=10; i++ ))  
do  
    echo $i  
done
```

- ▶ i ia pe rând toate valorile între 1 și 10
- ▶ în exemplu, blocul afișează \$i, dar pot apărea oricâte alte comenzi
- ▶ într-o singură linie:

```
for (( i=1; i<=10; i++ )); do cmd1 cmd2; done
```

- ▶ se comportă similar cu `switch` în C

```
case var in
  pattern1 )
    cmd1
    cmd2
    ;;
  pattern2 )
    cmd3
    ;;
  * )
    defaultcmd
    ;;
esac
```

- ▶ `pattern` – poate fi un string sau orice expresie de shell (similar cu wildcards-urile folosite pentru `grep(1)`)

Exemplu: case

Parsare subcomandă pentru un program SQL:

```
case "$1" in
  'CREATE')
    do_creat
    ;;
  'INSERT')
    do_insert
    ;;
  'SELECT')
    do_select
    ;;
  'USE')
    do_use
    ;;
  'DROP')
    do_drop
    ;;
  *)
    print_usage
esac
```

read

- ▶ citește una sau mai multe variabile din stdin
- ▶ sintaxă: `read var1 var2 ... varN`

```
$ read x y
1 2
$ echo $x $y
1 2
```

- ▶ dacă nu este dată nici o variabilă, se așteaptă să fie una singură și pune rezultatul în `$REPLY`

```
$ read
hello
$ echo $REPLY
hello
```

- ▶ citire line cu linie dintr-un fișier:
`cat foo.txt | while read LINE; do echo $LINE done`

Pentru a depana un script apelați-l cu shell-ul folosit și opțiunea -x.
Comenzile executate apar pe ecran prefixate cu + .

```
$ sh -x tolower.sh screaming.txt decent.txt
+ tr [A-Z] [a-z]
+ < screaming.txt
+ > decent.txt
```


Filtre

Utilizarea Sistemelor de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică
Email: paul.irofti@fmi.unibuc.ro

Vocabular

- ▶ `filtru` – program, comandă, operație care procesează ieșirea în format text a unei comenzi astfel încât noua formă să poată fi procesată mai departe de utilizator sau de un alt program
- ▶ `linie` – șir de caractere care se termină cu `\n` (sau `\r\n` în Windows)
- ▶ `separator` (sau `delimiter`) – caracter sau șir folosit pentru a delimita datele într-o linie
- ▶ `câmp` (*field*) – un subșir dintr-o linie care reprezintă un anumit tip de date
- ▶ exemplu:
 - ▶ `linie`: nume prenume grupă serie
 - ▶ `separator`: " "
 - ▶ `câmpuri`: nume, prenume, grupă și serie

cut(1)

cut(1) extrage câmpuri din fiecare linie primită la intrare

- ▶ `list` – numere sau intervale separate de virgulă sau spații
- ▶ `-b list` – lista conține poziții în bytes
- ▶ `-c list` – lista conține pozițiile caracterelor
- ▶ `-f list` – lista specifică câmpuri
- ▶ `-d delim` – specifică delimitatorul pentru câmpuri (implicit este `\t`)
- ▶ `-n` – nu împarte caractere multi-byte în bytes
- ▶ `-s` – sare peste liniile care nu conțin delimitatoare

Tipuri de apel

- ▶ `cut -b [-n] list [file ...]`
- ▶ `cut -c list [file ...]`
- ▶ `cut -f list [-s] [-d delim] [file ...]`

Exemple: cut(1)

Afișează numele și shellurile folosite de utilizatorii din sistem:

```
$ cut -d : -f 1,7 /etc/passwd
nobody:/sbin/nologin
paul:/bin/ksh
build:/bin/ksh
joe:/bin/ksh
_mysql:/sbin/nologin
_postgresql:/bin/sh
souuser:/bin/ksh
alex:/bin/ksh
```

Exemple: cut(1)

Arată numele și data la care s-au logat utilizatorii activi:

```
$ who | cut -c 1-8,18-30
deraadt  May  8 18:32
dlg      May  3 20:39
jsing    Apr 28 06:47
landry   Apr 19 14:22
deraadt  Apr 19 08:24
kettenis May  9 02:47
deraadt  May  3 22:18
pirofti  May  9 04:36
```

Lipește fișierele primite la intrare pe coloane (pe verticală)

- ▶ `-d list` – folosește delimitatorul pentru a înlocui caracterul linie nouă `\n` din fiecare fișier
- ▶ `-s` – serializare
- ▶ `-` – reprezintă intrarea standard (`stdin`)

Apel

- ▶ `paste [-s] [-d list] file ...`

Exemplu: paste(1)

Fie fişierele cu nume şi prenume:

```
$ echo "Paul\nAlex\nAna" > firstnames.txt
$ echo "Irofti\nAlexandrescu\nPopescu" \
    > lastnames.txt
```

Implicit, paste(1) lipeşte numele de prenume

```
$ paste firstnames.txt lastnames.txt
Paul      Irofti
Alex      Alexandrescu
Ana       Popescu
```

Acelaşi lucru dar cu serializare

```
$ paste -s firstnames.txt lastnames.txt
Paul      Alex      Ana
Irofti    Alexandrescu    Popescu
```

Exemplu: paste(1)

Afișează fișierele din directorul curent pe trei coloane:

```
$ ls | paste - - -  
1      10      11  
2       3       4  
5       6       7  
8       9      intro
```

Identic cu apelul `ls | paste -s -d '\t\t\n' -`

Crează o listă de directoare bin din sistem separate prin :

```
$ find / -name bin -type d | paste -s -d : -  
/usr/X11R6/bin:/usr/local/bin:  
/usr/local/lib/qt4/bin:  
...
```


split(1)

Împarte fișierul dat în mai multe fișiere de 1000 de linii fiecare.

- ▶ `-a suffix_length` – câte litere să conțină sufixul noilor fișiere
- ▶ `-b byte_count` – crează fișiere de lungimea dată în bytes
- ▶ `-l line_count` – crează fișiere cu numărul de linii dat
- ▶ `file` – fișierul de împărțit, implicit este `stdin`
- ▶ `name` – prefixul pentru noile fișiere

Apel

- ▶ `split [options] [file [name]]`

Implicit crează fișierele `xaa`, `xab`, `xac`, ...

Exemplu: split(1)

Împarte fișierul LaTeX în mai multe fișiere de 100 de linii:

```
$ wc -l uso-curs-11.tex
    362 uso-curs-11.tex
$ split -l 100 uso-curs-11.tex
$ ls x*
xaa xab xac xad
$ wc -l x*
    100 xaa
    100 xab
    100 xac
     62 xad
    362 total
```

Același lucru dar cu prefix uso-curs-11 și o singură literă sufix:

```
$ split -a 1 -l 100 uso-curs-11.tex uso-curs-11
```

join(1)

Alăturează linii care conțin chei identice din două fișiere diferite

- ▶ `file1`, `file2` – fișierele de intrare
- ▶ `-1 field` – câmpul cheie din fișierul 1
- ▶ `-2 field` – câmpul cheie din fișierul 2
- ▶ `-a file_number` – produce o linie pentru fiecare nepotrivire din fișierul dat (1 sau 2)

Apel

- ▶ `join [-1 field] [-2 field] [options] file1 file2`

Funcționează în aceeași paradigmă cu comanda SQL JOIN.

Exemplu: join(1)

Alătură persoanele pentru care am date legate de vârstă și venit:

```
$ cat age.txt
Paul 33
Alex 40
Ana 25
$ cat income.txt
Paul 3000
Ana 4500
$ join age.txt income.txt
Paul 33 3000
Ana 25 4500
```

Același lucru dar include și persoanele (ex. Alex) fără venit:

```
$ join -a1 age.txt income.txt
```

Basic Calculator – bc(1)

Calculator pentru operații aritmetice și logice

- ▶ -l – permite operații cu numere în virgulă mobilă
- ▶ -e *expr* – evaluează expresia, pot fi mai multe
- ▶ *file* – preia expresii din fișier
- ▶ operatorii binari sunt la fel ca cei din C
- ▶ operatorii logici &&, || și ! sunt disponibili în unele implementări dar nu sunt specificați de standardul POSIX
- ▶ reprezintă un limbaj de sine stătător cu blocuri de control (ex. *while*)
- ▶ implicit pornește un shell specializat și așteaptă comenzi

Apel

- ▶ `bc [-l] [-e expr] [file]`

bc(1) – funcții

- ▶ `s(x)` – sinus
- ▶ `c(x)` – cosinus
- ▶ `e(x)` – exponent
- ▶ `l(x)` – logaritm
- ▶ `a(x)` – arctangent
- ▶ `sqrt(x)` – radical
- ▶ `scale=n` – precizie, n numere zecimale
- ▶ `quit` – terminare program

Exemplu: bc(1)

Operații de bază:

```
$ echo "1/3" | bc
0
$ echo "1/3" | bc -l
.333333333333333333333333
$ echo "1>3" | bc -l
0
$ echo "1<3" | bc -l
1
$ bc -l -e "sqrt(2)" -e quit
1.41421356237309504880
```

Exemplu: bc(1)

```
$ bc -l
scale=4
sqrt(3)
1.7320
4*a(1)
3.1412
c(4*a(1))
-1.0000
c(4*s(1))
-.9750
quit
```


Translate – tr(1)

tr(1) traduce caractere primite din stdin și afișează rezultatul la stdout

- ▶ string1, string2 – caracterele din primul șir sunt traduse ca cele din al doilea string
- ▶ -C, -c – aplică complementul setului de caractere din string1
- ▶ -d – șterge caracterele ce apar în string1
- ▶ -s – elimină duplicatele traduse conform ultimul operand (fie string1 fie string2)

Tipuri de apel:

- ▶ tr [-Ccs] string1 string2
- ▶ tr [-Cc] -d string1
- ▶ tr [-Cc] -s string1

Exemplu: tr(1)

Crează o listă de cuvinte primite la intrare:

```
$ echo "Ana are mere" | tr -cs "[A-Za-z]" "\n"  
Ana  
are  
mere
```

Scrie textul primit cu majuscule

```
$ echo "Ana are mere" | tr "[a-z]" "[A-Z]"  
ANA ARE MERE
```

Obține șirurile de caractere dintr-un fișier binar:

```
$ echo "int main(){return 0;}" | cc -xc -o test -  
$ cat test | tr -cd "[:print:]"  
ELF>@@8@@@@>>>>>>HH H XX X @@TTTPtd44eHH H  
...
```

Stream EDitor – sed(1)

sed(1) este un editor de text în toată regula, mai mult este chiar un limbaj de programare!

- ▶ folosit în general pentru substituții și eliminări de text
- ▶ diferit de tr(1) folosește expresii regulate
- ▶ mod de funcționare
 - ▶ parcurge linie cu linie intrarea căutând un tipar
 - ▶ dacă a găsit un șir de caractere care respectă tiparul aplică funcția dată de utilizator asupra acestuia
 - ▶ funcțiile pot fi predefinite (substituție, eliminare) sau scrise de utilizator

Apel:

- ▶ `sed [options] command [file ...]`

Example: sed(1)

Substituție s,tipar,text,

```
$ echo "Paul Alex Ana" | sed s,Alex,Paul,  
Paul Paul Ana
```

```
$ echo "Paul Alex Alex Ana" | sed s,Alex,Paul,  
Paul Paul Alex Ana
```

Substituție s,tipar,text,flag

```
$ echo "Paul Alex Alex Ana" | sed s,Alex,Paul,g  
Paul Paul Paul Ana
```

```
$ echo "Paul Alex Alex Ana" | sed s,Alex,Paul,2  
Paul Alex Paul Ana
```

```
$ echo "Paul Alex Ana" | sed s,Alex,Paul,wfile  
Paul Paul Ana
```

```
$ cat file
```

```
Paul Paul Ana
```

Exemplu: sed(1)

Eliminarea linilor care conțin tiparul:

```
$ echo "Paul Alex\nAlex Ana" | sed /Paul/d  
Alex Ana
```

Dacă vrem să ștergem doar șirul de caractere:

```
$ echo "Paul Alex\nAlex Ana" | sed s/Paul//g  
Alex  
Alex Ana
```

Aplicare doar anumitor linii:

```
$ echo "Paul\nAlex\nAlex\nAna\nPaul\nGeorge" | \  
sed 1,3s/Paul//g
```

```
Alex  
Alex  
Ana  
Paul  
George
```

Exemplu: sed(1)

Inversarea a două cuvinte folosind expresii regulate

```
$ echo "Paul Irofti\nAlex Pop\nGeorge Stan" | sed 's
    ,^\([A-Z][A-Za-z]*\) \([A-Z][A-Za-z]*\),\2 \1,'
Irofti Paul
Pop Alex
Stan George
```

- ▶ ^ – caută un tipar la începutul liniei
- ▶ [A-Z] – trebuie să înceapă cu majusculă
- ▶ [A-Za-z]* – poate continua cu oricâte litere mici sau mari
- ▶ () – demarcă două expresii în tipar; folosim \ ca să nu fie interpretate de shell (*escaping*)
- ▶ \1 \2 – șirurile care au fost găsite cu cele două expresii

Exemple: sed(1)

Există și jocuri scrise în sed(1)



<http://aurelio.net/projects/sedarkanoid/>

Limbaj de programare specializat pentru procesarea datelor de tip text

- ▶ Alfred **A**ho, Peter **W**einberger și Brain **K**ernighan (1970)
- ▶ funcționează pe principiul identificare tipar - aplicare funcție ca `sed(1)`

- ▶ verifică mai mult de un tipar

```
tipar1 { comenzi }  
tipar2 { comenzi }
```

- ▶ verifică linia curentă cu fiecare tipar dat după care trece la următoarea linie
- ▶ Turing Complete

Apel:

- ▶ `awk [options] [program] file ...`

Exemple: awk(1)

Afișează linile care conțin tiparul

```
$ echo "Paul Irofti\nAlex Pop\nGeorge Stan" | awk '/  
    Pop/ {print}'  
Alex Pop
```

Găsește de câte ori apare un șir în fișier:

```
$ echo "Dori și Nemo\nNemo la dentist\nDori s-a  
    pierdut" | \  
> awk 'BEGIN {print "Finding Nemo and Dori"}  
> /[Nn]emo/ {nemo++}  
> /[Dd]ori/ {dori++}  
> END {print "Found Nemo " nemo " times and Dori "  
    dori " times!"}'  
Finding Nemo and Dori  
Found Nemo 2 times and Dori 2 times!
```

Comenzile marcate cu BEGIN și END se execută doar o dată.

Exemple: awk(1)

Procesarea câmpurilor dintr-un fișier

- ▶ \$0 – se referă la linia întreagă
- ▶ \$1,\$2,...,\$(10) – se referă la fiecare câmp în parte
- ▶ FS(*field separator*) – este *separatorul*; implicit setat ca spațiu

```
$ cat cont.txt
OP      SUM
IN      10
OUT     5
IN      20
IN      3
OUT     25

$ awk '
> BEGIN {print "Fonduri disponibile"; fonduri = 0}
> /IN/ { fonduri += $2 }
> /OUT/ {fonduri -= $2 }
> END {print fonduri "RON"}' cont.txt
Fonduri disponibile
3RON
```

awk(1): variabile și funcții

Variabile utile

- ▶ NF – numărul de câmpuri în linia curentă
- ▶ NR – numărul de linii citite până acum
- ▶ FILENAME – numele fișierului de intrare

Funcții utile

- ▶ toupper(), tolower() – litere mari, litere mici
- ▶ exp(), log(), sin() – funcții matematice
- ▶ length() – lungimea șirului
- ▶ int() – partea întreagă

Exemple: awk(1)

Afișează toate cuvintele:

```
$ echo "Ana are mere" | \
  awk '{for (i=1;i<=NF;i++) print $i }'
```

Ana
are
mere

Listă cu utilizatori și shell folosit:

```
$ awk 'BEGIN { FS = ":" }
> {print "User " $1 " uses " $7 " shell."}'
> /etc/passwd
```

User nobody uses /sbin/nologin shell.
User paul uses /bin/ksh shell.
User souuser uses /bin/ksh shell.
User _rsync uses /sbin/nologin shell.
User alex uses /usr/local/bin/bash shell.

Exemple: awk(1)

Afișează doar numele shellului pentru un utilizator anume:

```
$ awk 'BEGIN { FS = ":" }  
> /paul/ { cmd = "basename " $7;  
> cmd | getline shell;  
> print "User " $1 " uses " shell " shell."  
> close(cmd);  
> }' /etc/passwd  
User paul uses ksh shell.
```

Execută o comandă externă și obține rezultatul cu `getline`.

Funcții sistem

Utilizarea Sistemelor de Operare

Paul Irofti

Universitatea din București
Facultatea de Matematică și Informatică
Department de Informatică
Email: paul.irofti@fmi.unibuc.ro

Application Programming Interface (API)

- ▶ set de rutine, structuri de date și obiecte
- ▶ descrie comportamentul, intrările și ieșirile rutinelor
- ▶ unul sau mai multe protocoale de apel
- ▶ bazat pe cod sursă, nu cod compilat
- ▶ documentația unui API la un moment dat este însoțită de o versiune
- ▶ descris formal într-un *header* (ex. `xml.h`)
- ▶ de-a lungul timpului apar sau dispar elemente din API
- ▶ implementarea unui API este împachetată într-o bibliotecă
- ▶ exemplu: `libc.so.11.2`
- ▶ două implementări pentru aceeași versiune de API sunt interschimbabile fără nevoia altor operații (ex. recompilare, repornire)

Application Binary Interface (ABI)

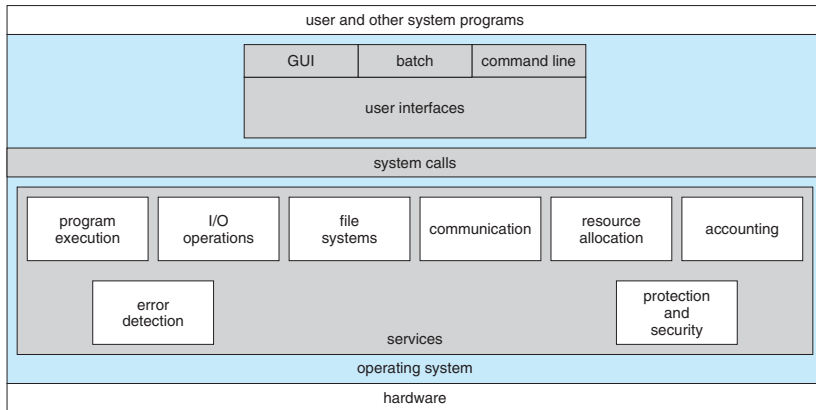
- ▶ asemănător cu principiile API
- ▶ bazat pe cod mașină (compilat), nu pe cod sursă!
- ▶ dependent de hardware
 - ▶ cum sunt împachetate și ordonate structurile de date
 - ▶ convenția de apel a funcțiilor (ex. unde sunt puse argumentele)
 - ▶ dimensiunea datelor (ex. `int` este `int32_t` sau `int64_t`)
 - ▶ aliniere la 16, 32 sau 64 de biți
- ▶ asigurat de compilatoare și sisteme de operare
- ▶ programe foarte vechi pot rula cu succes pe sisteme de operare moderne dacă ABI-ul a fost păstrat
- ▶ două programe binare pot comunica între ele fără acces la sursă dacă respectă același ABI
- ▶ cum rezolvă C++ problema ABI pentru funcții definite multiplu (*overloading*)?

Funcții sistem (syscalls)

API și ABI al sistemului de operare

- ▶ solicitarea unui serviciu din partea sistemului de operare
- ▶ arbitrează folosirea resurselor hardware și software între procese
- ▶ apelate cu ajutorul bibliotecilor de sistem (ex. biblioteca C)
- ▶ este interfața ce separă nivelul *userland* (al aplicațiilor) de sistemul de operare (nivelul *kernel*)
- ▶ API Win32 pentru Windows
- ▶ API POSIX pentru toate celelalte sisteme de operare: Linux, Android, Mac OS, OpenBSD etc.
- ▶ descrierea funcțiilor sistem se află în secțiunea 2 al manualelor Unix (ex. `man 2 open`)

Arhitectură și servicii



<http://codex.cs.yale.edu/avi/os-book/>

Exemplu: read(2)

```
ssize_t read(int d, void *buf, size_t nbytes);
```

read() folosește obiectul la care se referă descriptorul d din care încearcă să citească nbytes de date pentru a le scrie în bufferul indicat de buf

Întoarce numărul de bytes citați efectiv sau -1 în caz de eroare. Codul de eroare este pus în variabila globală errno.

Observații:

- ▶ d – poate fi orice obiect, nu doar un fișier
- ▶ buf – conține date binare; semantic sistemul de operare nu știe ce reprezintă; programul care a solicitat serviciul știe cum să le interpreteze
- ▶ nbytes – este o rugămințe, nu o cerere fermă, nu trebuie respectată

Exemplu: read(2)

Apel când vreau să citesc o cantitate fixă:

```
char buf[BUFSZ];
left = sizeof(buf);
nread = 0;
while (left) {
    nr = read(fd, buf, left);
    if (nr == -1 || nr == 0)
        break;
    nread += nr;
    left -= nr;
}
```

Apel când vreau să citesc tot:

```
while ((nr = read(fd, buf, sizeof(buf))) != -1 &&
        nr != 0) {
    /* consume nr bytes from buf */
}
```

Exemplu: read(2)

Senzor de umiditate

- ▶ d – reprezintă legătura dintre aplicație și senzor
- ▶ read(2) cere date kernel-ului care la rândul lui, prin driver, cere date senzorului
- ▶ stivă apel:
 1. aplicație
 2. kernel syscall (*upper layer*)
 3. driver senzor (*lower layer*)

Exemplu: read(2)

Fișier text

- ▶ d – reprezintă legătura dintre aplicație și disc
- ▶ read(2) cere date kernel-ului care la rândul lui, prin mai multe nivele de abstractizare, cere date mediului de stocare
- ▶ stivă apel:
 1. aplicație
 2. kernel syscall (*upper layer*)
 3. sistem de fișiere (*middle layer*)
 4. driver stocare – disc, usb, flash (*lower layer*)

Tipuri de funcții sistem

- ▶ control procese
- ▶ organizarea fișierelor
- ▶ manipularea dispozitivelor hardware
- ▶ configurarea sistemului
- ▶ comunicații
- ▶ protecție și securitate

Control procese

- ▶ crearea și oprirea proceselor
- ▶ obținerea și setarea atributelor unui proces
- ▶ suspendarea procesului pentru un interval de timp
- ▶ așteptarea ca un eveniment să aibă loc
- ▶ semnalarea că un eveniment a avut loc
- ▶ alocarea și eliberarea memoriei
- ▶ *dump* memorie proces în caz de eroare
- ▶ depanarea unui proces activ
- ▶ mecanisme de sincronizare și acces exclusiv la resurse

Exemple: funcții proces

```
pid_t fork(void);
```

Pornește un proces nou identic din momentul în care a fost apelat:

```
pid_t pid = fork();  
if (pid < 0)  
    return errno;  
else if (pid == 0)  
    /* child instructions */  
else  
    /* parent instructions */
```

fork(2) întoarce o valoare negativă dacă a eșuat. Dacă procesul a fost creat, părintele primește înapoi PID-ul copilului iar copilul primește valoarea zero.

Pentru sincronizare și ieșire se folosesc wait(2) și exit(3).

```
pid_t wait(int *status);  
void exit(int status);
```

Exemple: funcții proces

```
int exeve(const char *path, char *const argv[],
          char *const envp[]);
```

Pornește un proces nou și execută comanda din path cu argumentele argv.

```
pid_t pid = fork();
if (pid < 0)
    return errno;
else if (pid == 0)
    /* child instructions */
    char *argv[] = {"pwd", NULL};
    exeve("/bin/pwd", argv, NULL);
    perror(NULL);
else
    /* parent instructions */
```

Organizarea fișierelor

- ▶ crearea și ștergere fișierelor
- ▶ deschidere și închiderea unui fișier
- ▶ citire, scriere, poziționare într-un fișier
- ▶ obținerea și setarea proprietăților

Exemple: organizarea fișierelor

```
int open(const char *path, int flags, int mode);
```

Deschide un fișier din `path` în modul citire, scriere conform `flags` și dacă trebuie să-l creeze folosește drepturile `rxw` conform `mode`.
Întoarce un descriptor când este executat cu succes.

```
ssize_t write(int d, const void *buf, size_t nbytes);
```

Funcționează similar `read(2)`.

```
int close(int d);
```

Închide fișierul asociat descriptorului `d`.

Descriptori definiți implicit (ca cei din C)

- ▶ 0 – `stdin`
- ▶ 1 – `stdout`
- ▶ 2 – `stderr`

Manipularea dispozitivelor hardware

- ▶ cerere și eliberare acces la un dispozitiv
- ▶ citire, scriere, repoziționare
- ▶ obținerea și stabilirea diferitor proprietăți
- ▶ atașare, detașare dispozitive

Exemple: manipularea dispozitivelor hardware

Se folosesc tot `open(2)`, `read(2)`, `write(2)`, `close(2)`. În plus există `ioctl(2)` care are trei tipuri de apel

```
int ioctl(int d, unsigned long request);  
int ioctl(int d, unsigned long request, int arg);  
int ioctl(int d, unsigned long request, void *arg);
```

Dispozitivul asociat descriptorului `d` primește o cerere de tip `request` care are eventual și informație necesară în plus trimisă prin `arg`.

Exemple: manipularea dispozitivelor hardware

Operația `ioctl(2)` poate fi de tip intrare sau ieșire, caz în care al treilea argument este fie citit, fie scris. Tipul operației este dat implicit de numele cererii request.

Intrare:

```
char *font = "Comic Sans";  
ret = ioctl(fd, SETFONT, font);
```

Ieșire:

```
char product_key[24];  
ret = ioctl(fd, GETPRODKEY, product_key);
```

Configurarea sistemului

- ▶ informații privind ora și data
- ▶ setarea orei și a datei
- ▶ prelucrarea și configurarea proprietăților legate de procese, fișiere și dispozitive
- ▶ crearea, modificarea și eliminarea utilizatorilor și grupurilor
- ▶ configurarea și activarea serviciilor (ex. sshd, httpd)
- ▶ programarea executării unor comenzi la o anumită dată și oră
- ▶ stabilirea utilizării resurselor de către utilizatori (ex. disk quota)

Exemple: configurarea sistemului

Setarea orei și datei

```
int clock_gettime(clockid_t clock_id, struct timespec
    *tp);
int clock_settime(clockid_t clock_id, const struct
    timespec *tp);
int clock_getres(clockid_t clock_id, struct timespec *
    tp);
```

Obținerea PID-ului procesului curent și pe cel al procesului tată:

```
pid_t getpid(void);
pid_t getppid(void);
```

Semnaleză procesul după ce se scurge un timp dat

```
unsigned int alarm(unsigned int seconds);
```

- ▶ crearea și eliminarea unei conexiuni
- ▶ trimiterea și primirea mesajelor (ex. client-server)
- ▶ crearea și obținerea accesului la zone de memorie partajate (*shared-memory*)
- ▶ atașarea și detașarea dispozitivelor la distanță (ex. samba, nfs)
- ▶ configurarea dispozitivelor de rețea
- ▶ configurarea rutelor de comunicație
- ▶ filtrarea pachetelor și mesajelor

Exemple: comunicații

Conectează două procese prin conectarea descriptorului scriere (ieșire) a primului la descriptorul citire (intrare) al celui de-al doilea:

```
int pipe(int fildes[2]);
```

Este în general apelat înainte de o operație de tip `fork(2)`.

Partajarea memoriei cu `mmap(2)`

```
void * mmap(void *addr, size_t len, int prot, int  
            flags, int d, off_t offset);
```

Conținutul este luat prin intermediul descriptorului `d` începând de la poziția `offset`.

Exemple: comunicații

Crearea unui socket:

```
int socket(int domain, int type, int protocol)
```

Serverul deschide conexiunea:

```
int listen(int s, int backlog)
```

Clientul se conectează:

```
int connect(int s, const struct sockaddr *name,  
            socklen_t namelen);
```

Trimiterea și recepționarea mesajelor dintr-un socket:

```
ssize_t send(int s, void *msg, size_t len, int flags);  
ssize_t recv(int s, void *buf, size_t len, int flags);
```

- ▶ controlul și accesul la resurse
- ▶ setarea și obținerea drepturilor asupra obiectelor (ex. fișiere, dispozitive)
- ▶ permiterea și blocarea accesului utilizatorilor și grupurilor
- ▶ încapsularea și limitarea unui serviciu
- ▶ elevarea și retrogradarea drepturilor unui serviciu sau utilizator

Exemple: protecție și securitate

Schimbarea drepturilor de acces:

```
int chmod(const char *path, int mode);
```

Fișierul sau directorul se găsește în path (ex. /etc/passwd), iar tipul de acces în mode (ex. 0775).

Schimbarea proprietarului

```
int chown(const char *path, uid_t owner, gid_t group);
```

user și grup sunt întregii corespunzători ID-urilor din fișierele /etc/passwd și /etc/group.

Exemple: protecție și securitate

- ▶ încapsulare:
 - ▶ `chroot(2)`, `chroot(8)` – schimbă rădăcina
 - ▶ FreeBSD jail și sysjail: crearea unui mediu minimal pentru execuție
 - ▶ containers: solaris, cgroups, docker, vmm
- ▶ $W \wedge X$: write or execute, not both! (NX bit)
- ▶ ASLR: address space layout randomization
 - ▶ kernel
 - ▶ userland
 - ▶ libraries
- ▶ limitarea funcțiilor sistem ce pot fi apelate
 - ▶ pledge: promisiune de început ce tipuri de funcții sistem voi folosi în program
 - ▶ systrace: politică de acces la nivel de funcție
- ▶ SELinux, AppArmor: extinderea drepturilor Unix pentru a crea politici complexe de acces la date și resurse